

```

-----
include/linux/notifier.h
14 struct notifier_block
15 {
16 int(*notifier_call)(struct notifier_block *self, unsigned long, void *);
17 struct notifier_block *next;
18 int priority;
19 };
-----

```

`notifier_block` 包含指向函数 `notifier_call` 的指针，当事件发生时调用该函数，其参数包括指向保存信息的 `notifier_block` 对象的指针、相应事件代码或标志的值，以及指向特定于子系统的数据类型的指针。

`notifier_block` 结构还包含指向链中下一个 `notifier_block` 的指针和优先级声明。

例程 `notifier_block_register()` 和 `notifier_block_unregister()` 分别用于向特定通告程序链注册或注销一个 `notifier_block` 对象。

2.9 小结

本章阐述了许多探索 Linux 内核之前应当了解的背景知识，介绍了两种动态存储方法——链表和二叉搜索树，对这些数据结构的基本了解有助于读者理解进程和分页机制。接下来介绍了汇编语言的基础知识，以便在机器层进行探索和调试。同时，关于内联汇编程序，我们展示了同一函数中同时出现 C 语言代码和汇编代码的情况。最后，本章讨论了研究内核的各个方面所必需的各种命令和函数。

2.9.1 项目：Hellomod

本节介绍了一些基本概念，为稍后介绍的其他 Linux 相关概念和数据结构打下基础。该项目重点在于使用新的 2.6 驱动程序结构来创建一个可加载模块，并为随后的项目编译该模块。设备驱动程序非常复杂，因此，我们只介绍 Linux 模块的基本结构，在后面的项目中再来介绍这个驱动程序。该模块在 PPC 上和 x86 上都可以运行。

2.9.2 第一步：构造 Linux 模块的框架

我们写的第一个模块是基本的“hello world”字符设备驱动程序。首先，考虑该模块的基本代码，然后示范怎样使用新的 2.6 Makefile 系统（详情参见第 9 章）进行编译，最后，分别使用 `insmod` 命令和 `rmmmod` 命令来附加或移除^①该模块。

```

-----
hellomod.c
001
// hello world driver for Linux 2.6

004 #include <linux/module.h>

```

① 要确保在配置内核时允许卸载模块。

```

005 #include <linux/kernel.h>
006 #include <linux/init.h>
007 #MODULE_LICENSE("GPL"); //get rid of taint message

009 static int __init lkp_init( void )
{
    printk("<1>Hello,World! from the kernel space...\n");
    return 0;
013 }

015 static void __exit lkp_cleanup( void )
{
    printk("<1>Goodbye, World! leaving kernel space...\n");
018 }

020 module_init(lkp_init);
021 module_exit(lkp_cleanup);
-----

```

第4行：所有模块都要使用头文件 `module.h`，此文件必须被包含进来。

第5行：头文件 `kernel.h` 包含了常用的内核函数。

第6行：头文件 `init.h` 包含了宏 `__init` 和 `__exit`，它们允许释放内核占用的内存。建议浏览一下该文件中的代码和注释。

第7行：提示可能没有 GNU 公共许可证。有几个宏是在 2.4 版的内核中才开发的（详情参见 `modules.h`）。

第9~12行：这是模块的初始化函数，它必需包含要编译的代码、初始化数据结构等内容。第11行用函数 `printk()` 从内核发送消息，并提示加载模块后从何处读取该消息。

第15~18行：这是模块的退出和清理函数。此处可以放置终止该驱动程序时，所有执行相关清理工作的代码。

第20行：这是驱动程序初始化的入口点。对于内置模块，内核在引导时调用该入口点；对于可加载模块则在模块插入内核时才调用。

第21行：对于可加载模块，内核在此处调用 `cleanup_module()` 函数，而对于内置的模块，它没有什么作用。

在该驱动程序中，仅有一个初始化点（函数 `module_init`）和一个清理点（函数 `module_exit`）。加载或卸载模块时，内核会来寻找这些函数。

2.9.3 第二步：编译模块

如果你习惯使用老办法来编译内核模块（例如，从 `#define MODULE` 开始的模块），就会发现新的方法有很大变化。即使是首次编译 2.6 的模块，看起来也相当简单。该模块的 `Makefile` 文件基本内容如下：

`Makefile`

```
002 # Makefile for Linux Kernel Primer module skeleton (2.6.7)
```

```
006 obj-m += hellomod.o
```

要注意的是，需要向编译系统特别声明该模块要编译成可加载模块。该 Makefile 文件的命令行调用打包在名为 `doit` 的 `bash` 脚本文件中，如下所示：

```
-----doit
001 make -C /usr/src/linux-2.6.7 SUBDIRS=$PWD modules
-----
```

第 1 行：`-C` 选项告诉 `make` 程序读取 Makefiles 或做其他任何事之前，先要改变 Linux 源目录（本例中是 `/usr/src/linux-2.6.7`）。

执行 `./doit` 后可得到与以下内容类似的输出结果：

```
Lkp# ./doit
make: Entering directory '/usr/src/linux-2.6.7'
  CC [M]  /mysource/hellomod.o
  Building modules, stage 2
  MODPOST
  CC  /mysource/hellomod.o
  LD [M]  /mysource/hellomod.ko
make: Leaving directory '/usr/src/linux-2.6.7'
lkp# _
```

如果在 Linux 早期的版本上编译过或创建过 Linux 模块，那么此处还有一个链接步骤 `LD`，其输出模块是 `hellomod.ko`。

2.9.4 第三步：运行代码

现在我们已经准备好，可以将新的模块插入到内核中了。可以用命令 `insmod` 来实现，如下所示：

```
lkp# insmod hellomod.ko
```

`lsmod` 命令可以用于检查模块是否被正确插入到内核中了：

```
lkp# lsmod
Module      Size  Used by
hellomod    2696   0
lkp#
```

模块的输出由函数 `printk()` 来产生。该函数默认会输出信息到系统文件 `/var/log/messages` 中。要快速浏览这些消息可以输入如下命令：

```
lkp# tail /var/log/messages
```

这一命令打印日志文件的最后 10 行内容，可以看到我们的初始化信息：

```
...
...
Mar  6 10:35:55 lkp1 kernel: Hello,World! from the kernel space...
```

只要使用 `rmmmod` 命令，后面加上我们在 `insmod` 命令中看到的模块名，就可以从内核中移除

该模块（还可以看到退出时显示的信息）。如下所示：

```
lkp# rmmod hellomod
```

同样，输出的内容也在日志文件中，如下所示：

```
...  
...  
Mar  6 12:00:05 lkp1 kernel: Hello,World! from the kernel space...
```

根据 X 系统的配置或者是否有基本命令行，`printk` 的输出可以在控制台上显示，也可以存放在日志文件中。在下一个项目中，考虑系统的任务变量时会再提到这个问题。

2.10 习题

1. 描述散列表在 Linux 内核中如何实现。
2. 双向链表中的元素有一个 `list_head` 结构，内核采用该结构之前，它已有指向其他相似结构的 `prev` 指针和 `next` 指针。创建一个仅有 `prev` 和 `next` 指针的结构，其目的是什么？
3. 什么是内联汇编？为什么要使用它？
4. 假定要写一个访问串口寄存器的设备驱动程序。你会将这些地址标记为 `volatile` 吗？为什么？
5. 想象一下 `__init` 做些什么。你认为什么类型的函数会使用这个宏？



进程：程序执行的基本模型

本章内容

- 程序
- 进程描述符
- 进程的创建：系统调用 `fork()`、`vfork()` 和 `clone()`
- 进程的生命周期
- 进程的终止
- 了解进程的动态：调度程序的基本构架
- 等待队列
- 异步执行流程

进程 (process) 是程序执行的基本单位，是了解操作系统工作原理的重要概念。非常有必要理解进程和程序之间的差异。程序指的是由若干函数组成的可执行文件，而进程指的是特定程序的一个实例。进程是对硬件所提供的资源进行操作的基本单元，也是顺序执行其实例化程序的基本单位。操作系统根据进程的需要管理和使用系统资源。

计算机能做很多事情，而这最终都是由进程来实现的。进程能够执行的任务包罗万象，上至用户命令的执行，下至系统资源的管理和硬件的访问。在一定程度上，进程是由它要执行的一组指令、寄存器的内容和程序执行时程序计数器以及它们的状态定义的。

进程与任何动态实体类似，会经历各种状态。事实上，每个进程都有一个生命周期：进程被创建后，将存活一段不定长的时间，在此期间，它将经历各种状态的改变，直到最后消亡。图 3-1 大体上显示了进程的生命周期。

当 Linux 系统加电时，它所需要的进程数是未知的。操作系统在运行的过程中将按照实际需要来创建或撤销进程。

进程是由当前已有的进程调用 `fork()` 来创建的。分叉的进程叫做**子进程** (child process)，创建者叫做**父进程** (parent process)。子进程和父进程继续并发运行。如果父进程继续繁衍更多的子进程，那么新创建的这些进程就是第一个子进程的**兄弟进程** (sibling process)。子进程们还可以依次繁衍出它们自己的子进程。这样就建立起定义关系的进程之间的一种层次关系。

进程被创建后，就准备变为**运行进程** (running process)。这就意味着内核已经为 CPU 执行进程建立起所有的结构，并获取所有必要的信息。当一个进程准备变为运行进程但还没有被选中运

行时，它处于就绪状态。当它变为运行进程后，就能够：

- 被“取消”，被调度程序置为就绪状态(ready)；
- 被中断，并转入等待或阻塞(blocked)状态；
- 变为僵死(zombie)状态，踏上进程消亡之路。这可以通过调用 `exit()` 来实现。

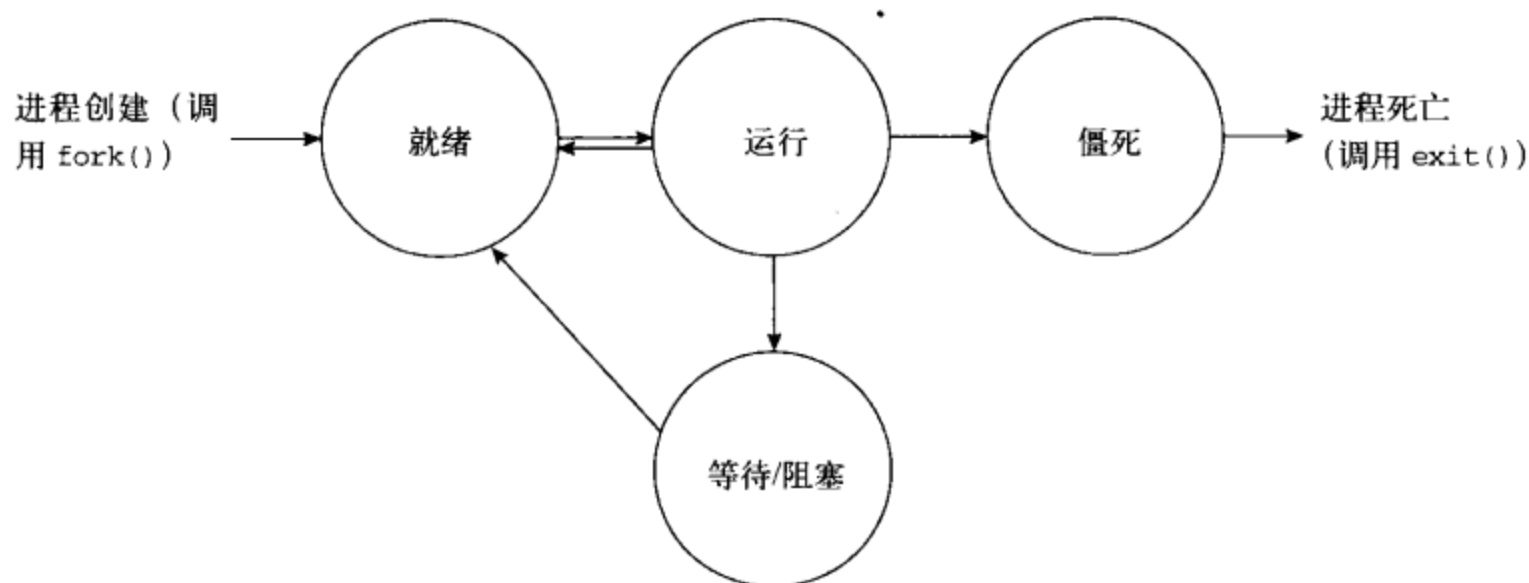


图 3-1 进程的生命周期

本章将详细考察所有这些状态及状态间的转换。调度程序负责选中或取消 CPU 将要执行的进程，我们在第 7 章再详细地讨论调度程序。

程序包含位于内存的多个组成部分，执行程序的进程将根据需要来访问这些内容，包括文本段(text segment)、数据段(data segments)、栈(stack)和堆(heap)。文本段存放 CPU 所执行的指令；数据段存放进程操作的所有数据变量；栈存放自动变量和函数数据；堆存放动态内存分配的数据。当进程被创建时，子进程收到父进程的数据副本，包括数据空间、堆、栈和进程描述符。下一节将详细介绍 Linux 的进程描述符。

人们可以采用多种方式来解释进程，而本书是从进程执行的高层面开始，然后追踪到内核层面，其间穿插说明内核对它所给予的支持。

作为程序员，我们熟悉编写、编译和执行程序。但是，如何把这些与进程联系起来呢？本章将讨论一个示例程序，并从该程序的创建开始，追踪其某些关键任务的执行。在本例中，Bash shell 进程创建的进程将实例化该示例程序；接下来，再由示例程序实例化另外一个子进程。

在继续讨论进程之前，需要解释几个命名约定。通常，术语进程和任务指的是同一件事。当说到运行进程时，就是指 CPU 当前正在执行的进程。

用户态与内核态的比较

我们说一个程序正在用户态(User Mode)下运行或正在内核态(Kernel Mode)下运行，这到底意味着什么呢？在进程的生命周期中，它或者执行自己的代码，或者执行内核代码。所谓内核代码是指当系统调用被执行时、异常发生时或者中断到来时(在中断处理程序中)执行的代码。进程使用的代码如果不是系统调用，就是用户态的代码，因此，进程正运行在用户态，

也服从处理器对它所施加的限制。如果进程正在执行系统调用中，我们说它正运行在内核态。从硬件的角度来看，Intel 处理器上的内核代码就是运行在 0 环上，PowerPC 上的内核代码就是运行在超级用户模式下。

3.1 程序

本节引入一个简单的示例程序，叫做 `create_process`。这个 C 程序阐明了进程可能经历的各种状态、系统调用（这将会导致进程在不同状态间转换）以及对支持进程执行的内核对象的操作，其目的是帮助大家理解程序是如何被实例化成进程的，操作系统又是如何处理进程的。

```
-----
create_process.c
1  #include <stdio.h>
2  #include <sys/types.h>
3  #include <sys/stat.h>
4  #include <fcntl.h>
5
6  int main(int argc, char *argv[])
7  {
8      int fd;
9      int pid;
10
11
12      pid = fork();
13      if (pid == 0)
14      {
15          execl("/bin/ls", NULL);
16          exit(2);
17      }
18
19      if(waitpid(pid) < 0)
20          printf("wait error\n");
21
22      pid = fork();
23      if (pid == 0){
24          fd=open("Chapter_03.txt", O_RDONLY);
25          close(fd);
26      }
27
28      if(waitpid(pid)<0)
29          printf("wait error\n");
30
31
32      exit(0);
33  }
-----
```

这个程序定义了一个**执行上下文**（context of execution），它包括满足程序定义的要求所需资源的相关信息。例如，在任一时刻，CPU 只严格执行一条从内存取得的指令^①。但是，如果没有围绕这条指令的上下文，就无从得知所引用的指令与程序逻辑之间究竟有着怎样的关系，那么，这条指令也就没有任何意义。进程的上下文由程序计数器、寄存器、内存和文件（或者所访问的

① 回想一下以前提到的代码段。

硬件)中存储的值组成。

这个程序编译并链接后产生一个可执行文件,其中存放着执行这个程序所需要的全部信息。本书第9章详细讨论了程序地址空间的划分,以及当讨论进程映像和二进制格式时,地址空间的划分和程序所引用的信息有怎样的关系。

进程具有很多特征,这些特征不仅描述了进程本身,还使它有别于其他进程。进程管理所必需的特征存放在一个单独的数据类型中,那就是所谓的**进程描述符**。在深入研究进程管理之前,有必要讨论一下进程描述符。

3.2 进程描述符

在内核中,进程描述符是一个名为 `task_struct` 的结构体,用于存放进程的属性和信息,与进程相关的所有内核信息都存储在这个结构体中。在其生命周期内,进程要与内核的很多方面——如内存管理和进程调度等打交道,因此,进程描述符除了记录 UNIX 进程的标准属性外,还要记录上述交互过程的相关信息。内核采用循环双向链表 `task_list` 来存放所有进程描述符,并借助全局变量 `current` 来存放当前运行进程的 `task_struct` 的引用。(本书中提及 `current` 时,指的就是当前运行进程的进程描述符。)

进程可能由一个或多个线程组成,每个线程都对应一个 `task_struct`,其中包括一个唯一的线程 ID。在一般的进程中,线程共享相同的内存地址空间。

以下是在进程的生命周期中,进程描述符必须记录的信息:

- ☐ 进程的属性;
- ☐ 进程间的关系;
- ☐ 进程的内存空间;
- ☐ 文件管理;
- ☐ 信号管理;
- ☐ 进程的信任状;
- ☐ 资源限制;
- ☐ 与调度相关的字段。

接下来详细讨论 `task_struct` 结构体中的各个字段。本节将描述这些字段的作用以及与之相关的实际处理过程。虽然许多字段用于描述与上述各类信息相关的活动,但某些内容已经超出本书的范围。`task_struct` 结构的定义位于 `include/linux/sched.h` 中。

```
-----
include/linux/sched.h
384  struct task_struct {
385      volatile long state;
386      struct thread_info *thread_info;
387      atomic_t usage;
388      unsigned long flags;
389      unsigned long ptrace;
390
391      int lock_depth;
```



```
392
393     int prio, static_prio;
394     struct list_head run_list;
395     prio_array_t *array;
396
397     unsigned long sleep_avg;
398     long interactive_credit;
399     unsigned long long timestamp;
400     int activated;
401
402     unsigned long policy;
403     cpumask_t cpus_allowed;
404     unsigned int time_slice, first_time_slice;
405
406     struct list_head tasks;
407     struct list_head ptrace_children;
408     struct list_head ptrace_list;
409
410     struct mm_struct *mm, *active_mm;
411     ...
413     struct linux_binfmt *binfmt;
414     int exit_code, exit_signal;
415     int pdeath_signal;
416     ...
419     pid_t pid;
420     pid_t tgid;
421     ...
426     struct task_struct *real_parent;
427     struct task_struct *parent;
428     struct list_head children;
429     struct list_head sibling;
430     struct task_struct *group_leader;
431     ...
433     struct pid_link pids[PIDTYPE_MAX];
434
435     wait_queue_head_t wait_chldexit;
436     struct completion *vfork_done;
437     int __user *set_child_tid;
438     int __user *clear_child_tid;
439
440     unsigned long rt_priority;
441     unsigned long it_real_value, it_prof_value, it_virt_value;
442     unsigned long it_real_incr, it_prof_incr, it_virt_incr;
443     struct timer_list real_timer;
444     unsigned long utime, stime, cutime, cstime;
445     unsigned long nvcs, nivcs, cnvcs, cnivcs;
446     u64 start_time;
447     ...
450     uid_t uid, euid, suid, fsuid;
451     gid_t gid, egid, sgid, fsgid;
452     struct group_info *group_info;
453     kernel_cap_t cap_effective, cap_inheritable, cap_permitted;
454     int keep_capabilities:1;
455     struct user_struct *user;
```

```

...
457     struct rlimit rlim[RLIM_NLIMITS];
458     unsigned short used_math;
459     char comm[16];
...
461     int link_count, total_link_count;
...
467     struct fs_struct *fs;
...
469     struct files_struct *files;
...
509     unsigned long ptrace_message;
510     siginfo_t *last_siginfo;
...
516 };

```

3.2.1 与进程属性相关的字段

进程的属性分类是我们为进程特征定义的包罗万象的分类，这些特征与进程的状态和标识相关。任何时候检查这些域的值均为内核黑客提供了获取进程当前状态的机会。图 3-2 说明了 task_struct 中与进程属性相关的域。

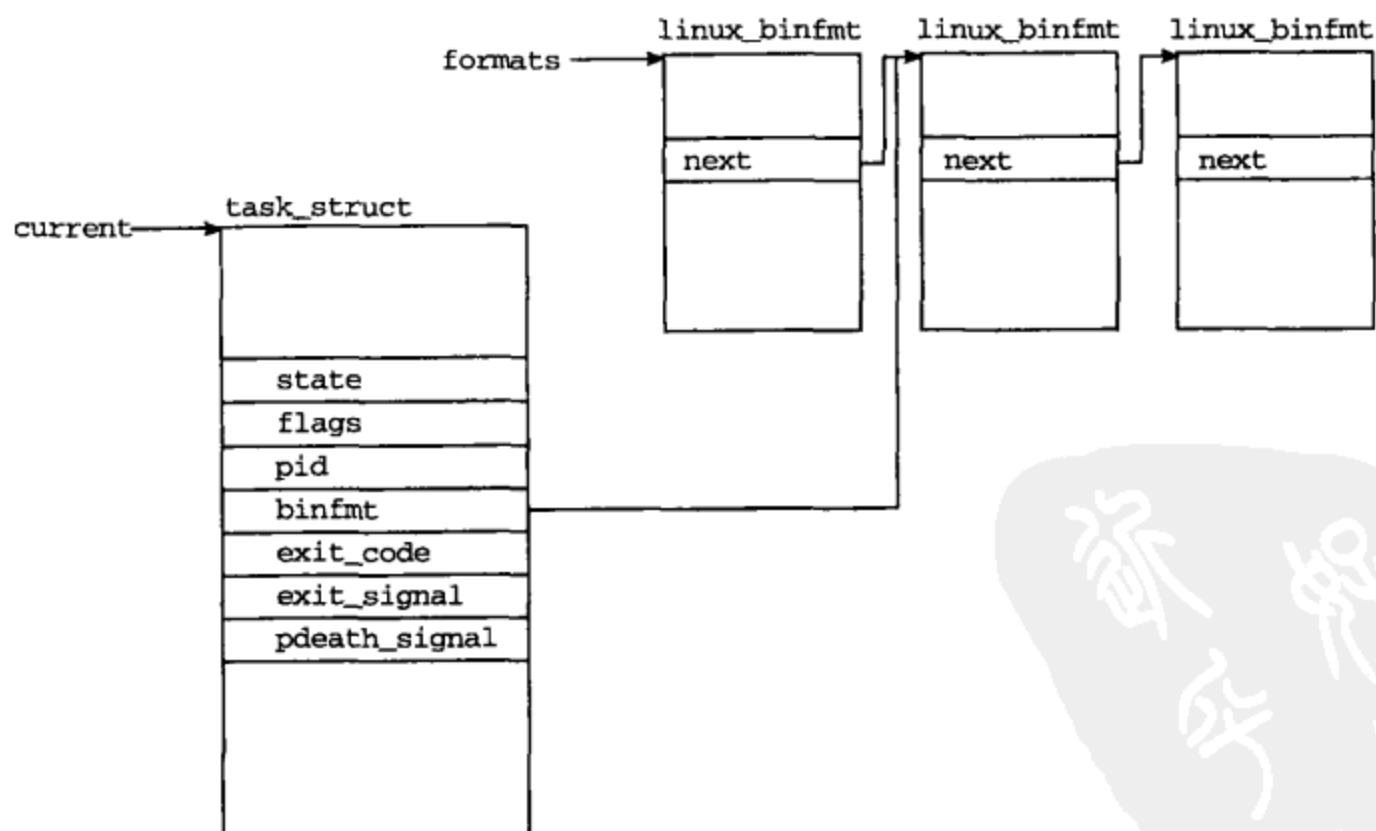


图 3-2 与进程属性相关的字段

1. state

State 字段用于记录进程的状态，在其执行生命周期内，进程可以从该域中找到自己所处的状态，其可能的值有 `TASK_RUNNING`、`TASK_INTERRUPTIBLE`、`TASK_UNINTERRUPTIBLE`、`TASK_ZOMBIE`、`TASK_STOPPED` 以及 `TASK_DEAD`（详见本章 3.4 节）。

2. pid

在 Linux 中,每个进程都有唯一的进程标识符 (process identifier) `pid`。`pid` 位于 `task_struct` 结构体中,类型为 `pid_t`,若追本溯源的话,`pid_t` 是整数类型,但 `pid` 默认的最大值是 32 768 (短整型的最大可能值)。

3. 标志

标志定义的是进程的特殊属性。每个进程的标志都是在 `include/linux/sched.h` 中定义的,表 3-1 给出了这些标志的值,它们为内核黑客提供了进程所处状态的更多相关信息。

表3-1 `task_struct`结构体中标志字段的部分值

Flag名称	何时设置
<code>PF_STARTING</code>	进程创建时设置
<code>PF_EXITING</code>	调用 <code>do_exit()</code> 时设置
<code>PF_DEAD</code>	进程退出、调用 <code>exit_notify()</code> 时设置。此时,进程的状态是 <code>TASK_ZOMBIE</code> 或 <code>TASK_DEAD</code>
<code>PF_FORKNOEXEC</code>	父进程创建子进程时设置该标志

4. binfmt

Linux 支持多种可执行文件格式。可执行文件格式是为指明程序代码如何被载入内存而定义的一种结构。图 3-2 说明了 `task_struct` 与 `linux_binfmt` 结构体之间的关系。其中,`linux_binfmt` 结构体包含了与特定二进制格式相关的所有信息 (详见第 9 章)。

5. exit_code和exit_signal

`exit_code` 与 `exit_signal` 字段分别存放进程的退出值和终止信号 (如果使用了该字段的话)。这是将子进程的退出值传给其父进程的方式。

6. pdeath_signal

`pdeath_signal` 是父进程消亡时发出的信号。

7. comm

通常通过在命令行调用一个可执行程序来创建进程。调用时,`comm` 字段用于存放该可执行程序的名称。

8. ptrace

当进程因进行性能测定而调用系统调用 `ptrace()` 时设置 `ptrace` 字段。`ptrace()` 的标志在文件 `include/linux/ptrace.h` 中定义。

3.2.2 与调度相关的字段

进程运行时宛如拥有自己的虚拟 CPU,事实上是多个进程共享一个 CPU。为了在运行的进程之间进行切换,每个进程均与调度程序密切相关 (详见第 7 章)。

然而,要理解上述的某些字段,势必要理解调度的基本概念。当多个进程都准备好等待运行时,由调度程序决定谁先运行、运行多长时间。调度程序通过给每个进程分配一个时间片和优先级来实现公平、高效的调度。时间片定义了一个进程被切换掉,运行另一个进程之前允许运行的

时间长度。进程的优先级则是一个数值，它定义了相对其他就绪进程而言，进程被允许执行的相对顺序，优先级越高的进程会越早被调度运行。图 3-3 所示的字段存放了调度所需的值。

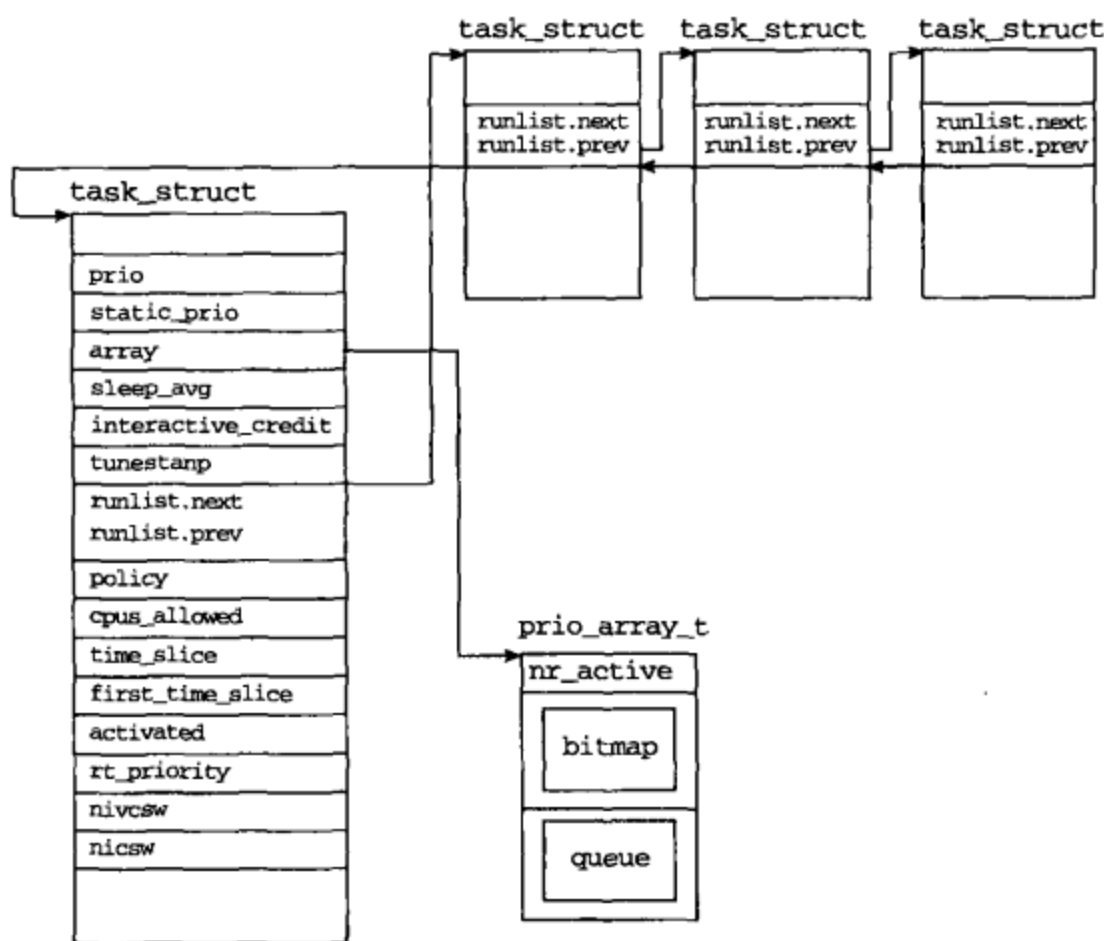


图 3-3 与调度相关的字段

1. prio

在第 7 章，将会看到进程的动态优先级是由进程的调度历史及一个具体的 `nice` 值确定的值（有关 `nice` 值的更多说明详见下面的注释框）。当进程时间片用完且进程未被执行时，其动态优先级将在睡眠时被更新，其值 `prio` 与下面将要说明的 `static_prio` 字段的值有关。`prio` 的值是在 `static_prio` 的基础上加/减 5，具体情况依据进程的历史而定。如果已经睡眠了很长时间的话，那么它将得到 +5 的奖励；反之，如果运行了相当长时间且用完了其时间片，那么它将得到 -5 的惩罚。

2. static_prio

`static_prio` 的值等于 `nice` 的值，前者的默认值为 `MAX_PRIO-20`。在我们的内核中，`MAX_PRIO` 默认为 140。

nice

系统调用 `nice()` 允许用户修改进程的静态调度优先级。`nice` 值的变化范围是 -20~19。函数 `nice()` 调用 `set_user_nice()` 来设置 `task_struct` 中的 `static_prio` 字段。`static_prio` 的值根据 `nice` 值利用宏 `PRIO_TO_NICE` 来计算。同样，`nice` 的值是根据 `static_prio` 值通过调用宏 `NICE_TO_PRIO` 来计算的。

```
-----kernel/sched.c
#define NICE_TO_PRIO(nice) (MAX_RT_PRIO + nice + 20)
#define PRIO_TO_NICE(prio) ((prio - MAX_RT_PRIO - 20)
```

3. **run_list**

run_list 指向 **runqueue**。**runqueue** 是一个包含所有准备运行的进程的队列，详见 3.6.1 节中的 **runqueue** 结构。

4. **array**

array 指向 **runqueue** 的优先级数组。3.6 节将详细阐述该数组的相关内容。

5. **sleep_avg**

sleep_avg 用于计算进程的有效优先级，它是进程睡眠过程耗费的时钟周期的平均数。

6. **timestamp**

timestamp (时间戳) 用于当进程睡眠或放弃处理器时计算 **sleep_avg**。

7. **interactive_credit**

interactive_credit 与 **sleep_avg**、**activated** 共同使用来计算 **sleep_avg** 的值。

8. **policy**

policy 用于确定进程的类型，例如，分时进程和实时进程。进程的类型极大地影响到其调度优先级。有关该字段的内容，详情请参考第 7 章。

9. **cpus_allowed**

cpus_allowed 指明由哪个 CPU 来处理任务。在多处理器系统中，可通过这种方式指定特定的任务在哪个 CPU 上运行。

10. **time_slice**

time_slice 定义进程每次被调度时允许运行的最长时间。

11. **first_time_slice**

first_time_slice 被反复设置为 0 并记录调度时间。

12. **activated**

activated 记录平均睡眠时间的增减。若一个不可中断的睡眠进程被唤醒，该字段将被置为 -1。

13. **rt_priority**

rt_priority 是实时进程的优先级，是一个静态值，只能通过 **schedule()** 来更新，支持实时任务时一定会用到该字段。

14. **nivcsw**和**nvcs**

系统中存在不同类型的上下文切换，内核将记录这些信息，以备分析。全局上下文切换计数器将根据上下文切换中涉及的不同转换种类，被设置成 4 种不同的上下文切换计数器中的一个 (详情见 7.1.2 节)。基本上下文切换计数器主要有两种。

- ❑ **nivcsw** (被动的上下文切换次数) 字段记录进程被内核抢占的次数，其值仅当进程从内核抢占代码返回时才增加，此时上下文切换计数被置为 **nivcsw**。
- ❑ **nvcs** (主动的上下文切换次数) 字段记录除去内核抢占之外的上下文切换次数。若前一

个状态不是主动抢占的话，则上下文切换计数被置为 `nvcs`。

3.2.3 涉及进程间相互关系的字段

结构体 `task_struct` 中的下列字段涉及进程间的相互关系。每个进程 `p` 都有一个创建它的父进程。进程 `p` 本身也可以创建其他进程，因此，它可能有子进程。由于进程 `p` 的父进程可以创建多个进程，进程 `p` 也可能有兄弟进程。图 3-4 给出了这些进程的 `task_struct` 之间有怎样的关系。

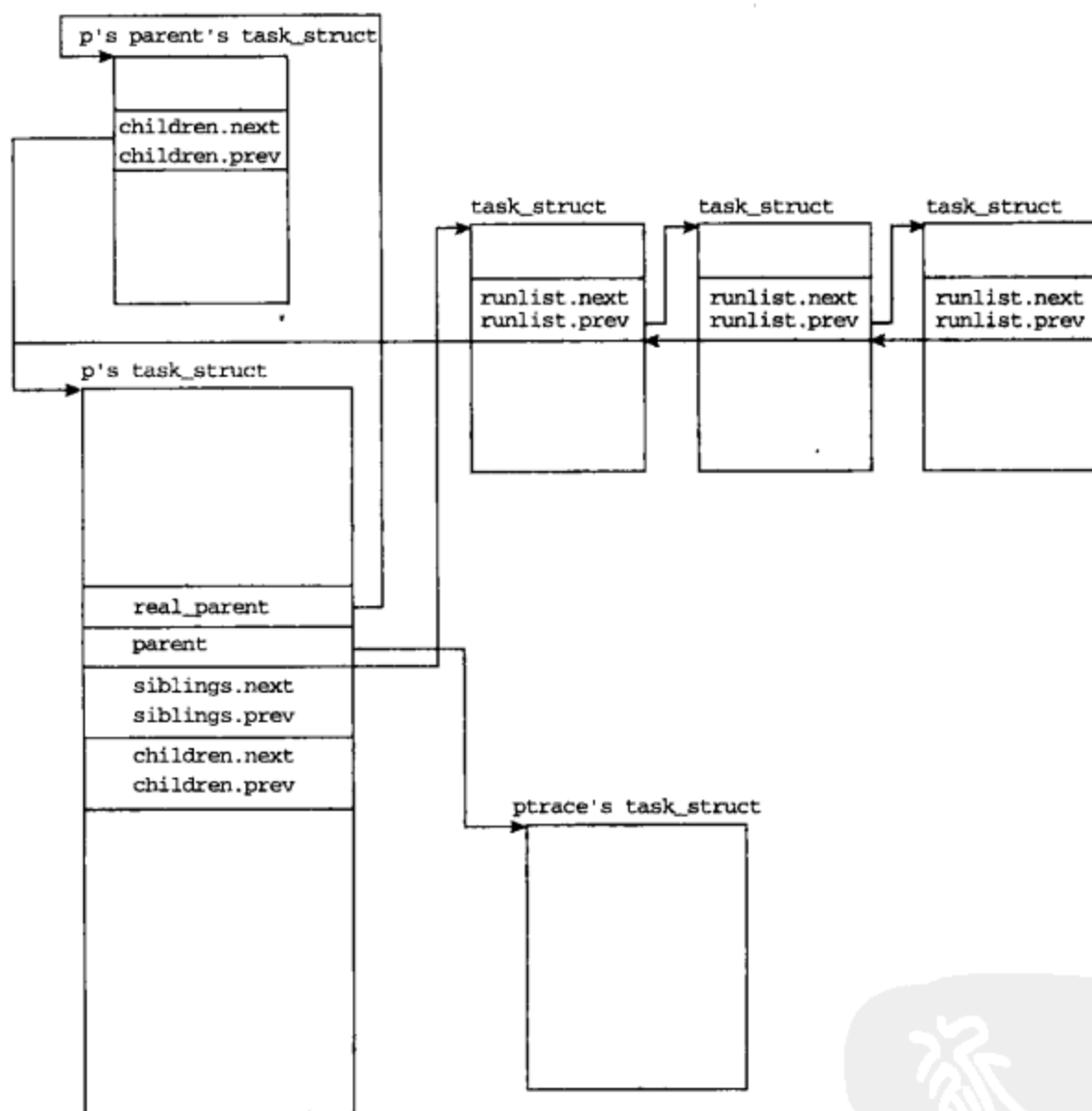


图 3-4 涉及进程间相互关系的字段

1. `real_parent`

`real_parent` 指向当前进程的父进程的进程描述符。若当前进程的父进程已经消亡，那么它指向 `init()` 的进程描述符。在早期的内核版本中，该字段是 `p_opptr`。

2. `parent`

`parent` 是指向其父进程描述符的指针。在图 3-4 中，该指针指向 `ptrace` 的 `task_struct`。当 `ptrace` 在某一进程上运行时，`task_struct` 中的 `parent` 字段指向 `ptrace` 进程。

3. `children`

`children` 是指向当前进程的子进程列表的 `struct`。

4. sibling

sibling 是指向当前进程的兄弟进程列表的 struct。

5. group_leader

进程可以是某一进程组中的成员。每个进程组都有一个被定义为组长的进程。如果进程是某个组的成员，则 group_leader 是一个指向其组长的进程描述符的指针。组长进程通常拥有自己的 tty，即控制终端，进程就是从这个终端创建的。

3.2.4 与进程信任状相关的字段

在多用户系统中，必须能够区分进程究竟是由哪个用户创建的，这一点从系统的安全性和保护用户数据的角度来讲也是必要的。因此，每个进程都有一个信任状，系统根据这个信任状来确定进程可以访问和不可以访问的系统资源。图 3-5 解释了 task_struct 结构体中与进程信任状有关的字段。

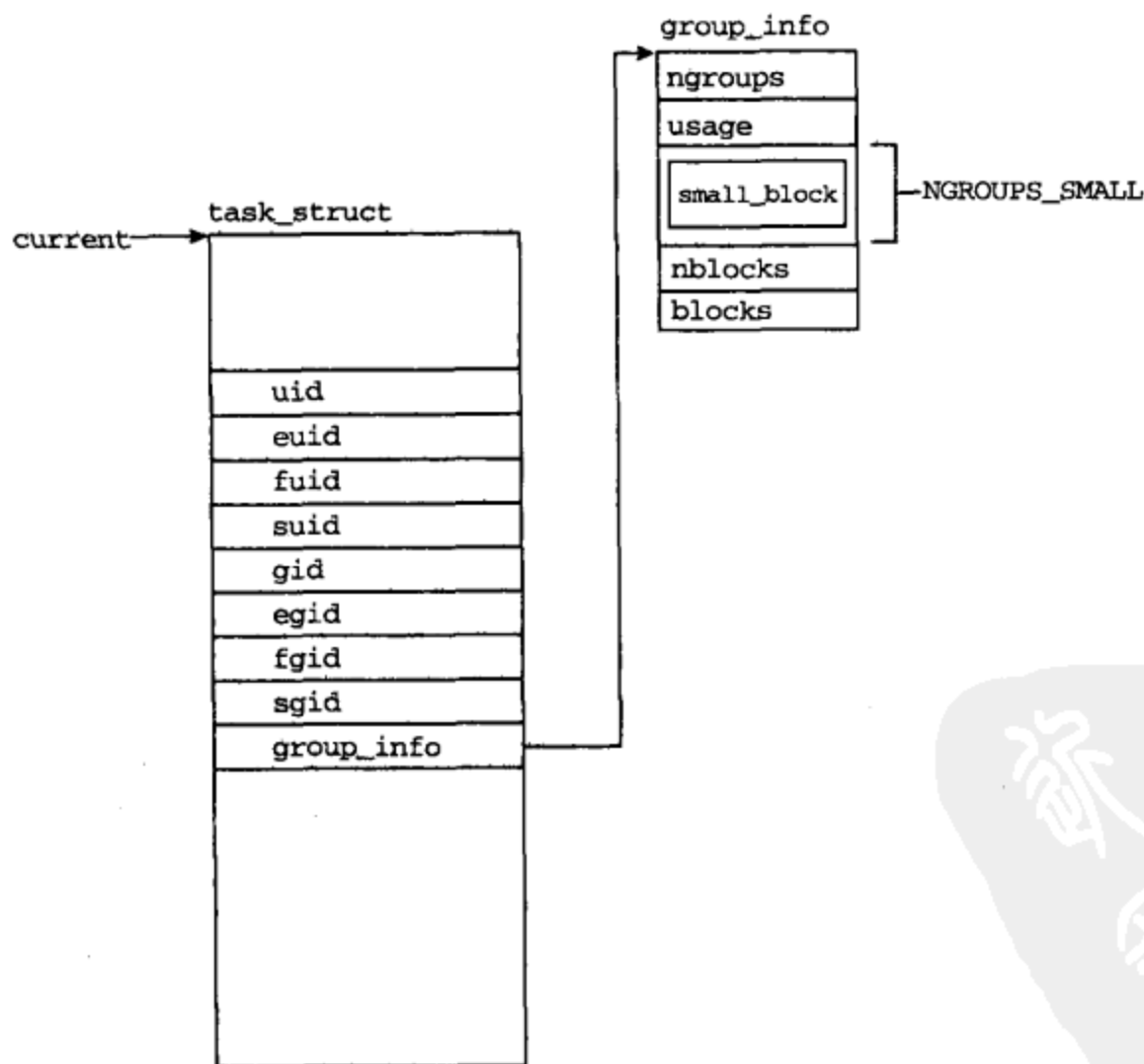


图 3-5 与进程信任状相关的域

1. uid和gid

uid 用于存放创建该进程的用户的 ID，该字段是出于数据保护和系统安全方面的考虑而设置的。同理，gid 是该进程所在进程组的组 ID。进程 0 的 uid 和 gid 分别是 root 的用户 ID 和组 ID。

2. **uid**和**legid**

有效用户 ID 通常存放着和 **uid** 字段相同的值,但如果被执行的程序已将设置 UID 的位(SUID)置位的话,情况就有所不同了。此时,有效用户 ID 指的是程序文件拥有者的 ID。通常,这允许任意用户能够以与另一个用户(比如说, **root**)相同的权限来运行一个特殊的程序。有效组 ID 的作用与之类似,也是只有当设置组 ID 的位(SGID)被置位后,其值才会与 **gid** 不同。

3. **suid**和**sgid**

suid (存放用户 ID) 和 **sgid** (存放组 ID) 用于系统调用 **setuid()**。

4. **fsuid**和**fsgid**

文件系统检测时将专门检查 **fsuid** 和 **fsgid** 的值。它们通常和 **uid**、**gid** 的值相同,除非调用了 **setuid()**。

5. **group_info**

在 Linux 中,一个用户可以是多个组的成员。这些组可能拥有不同的访问系统和存取数据的权限。因此,进程必须继承其信任状。**group_info** 字段是一个指向 **group_info** 类型的结构体的指针,结构体 **group_info** 中存放的是进程所在的不同组的所有信息。

结构体 **group_info** 允许进程在有充足的可用内存时与多个组关联。在图 3-5 中,**group_info** 的字段 **small_block** 就是一个类型为 **gid_t** 的数组,数组中有 **NGROUPS_SMALL** (本例中为 32) 个 **gid_t** 单元。如果某个进程所属的组超过 32 个,内核可以为超出 **NGROUPS_SMALL** 的部分分配块或者页来存放所需的 **gid_t** 数目。**nblocks** 存放已分配的块的数目,而 **ngroups** 存放 **small_block** 数组中单元的个数,该数组的每个单元中存放的都是 **gid_t** 的值。

3.2.5 与进程权能相关的字段

一般来说,UNIX 系统通过为每个给定进程赋予特权级(超级用户或 UID=0 的用户)或非特权级(其他任何进程),从而对特定的访问和操作提供与进程相关的保护。在 Linux 中,引入权能来划分以前只有超级用户才能执行的操作,换句话说,权能就是能授予进程的一种或多种特权,这种“授予”彼此相互独立且不依赖于进程的 UID。这样一来,特殊的进程可以获得一些权限来执行某些特殊的管理任务,而不必取得所有特权,也无需成为超级用户的进程。因此,权能被定义为给定的管理操作。图 3-6 给出了与进程权能相关的字段。

cap_effective、**cap_inheritable**、**cap_permitted**和**keep_capabilities**

用于支持权能模型的结构是一个 32 位的无符号数,是在文件 **include/linux/security.h** 中定义的。每个 32 位的掩码对应一个权能集,每种权能占用一个二进制位。

□ **cap_effective**: 有效权能。这是当前能够被进程使用的权能。

□ **cap_inheritable**: 可继承权能。这是可以通过系统调用 **execve** 来传递的权能。

□ **cap_permitted**: 许可权能。这可以是有效的权能或者可继承的权能。

要理解上述 3 种类型权能之间的区别,方法之一是考虑与其父进程创建的基因池类似的许可权能。考虑到父进程创建的基因质量,可以只显示其中一个子集(即有效权能),或者传递这个子集(即可继承权能)。许可权能构成了一种可能性,而有效权能把这种可能性变成了现实。

因此, **cap_effective** 和 **cap_inheritable** 永远是 **cap_permitted** 的子集。

□ `keep_capabilities`: 当调用 `setuid()` 时, 记录进程到底是放弃还是保持其权能。表 3-2 列出了一部分在 `include/linux/capability.h` 中定义的权能。

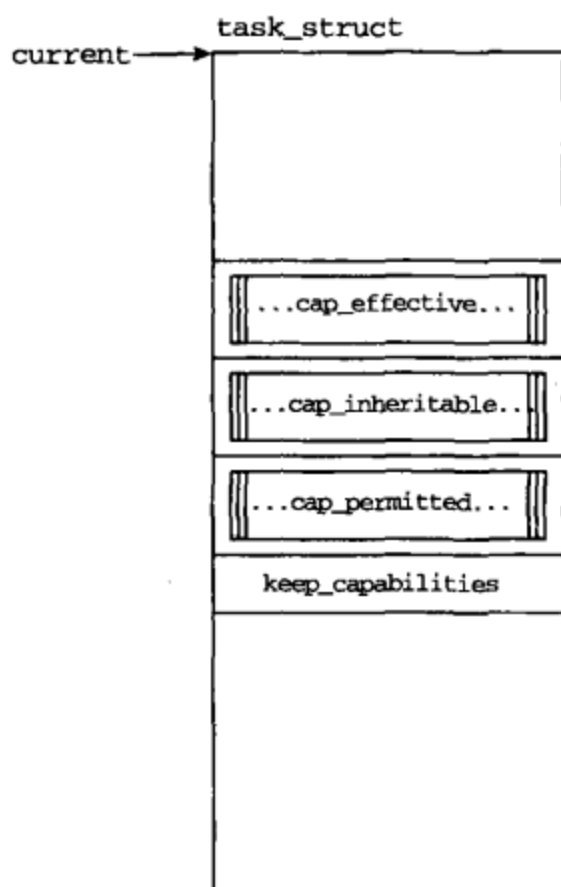


图 3-6 与进程权能相关的字段

表3-2 部分权能

权 能	说 明
<code>CAP_CHOWN</code>	忽略由 <code>chown()</code> 强加的限制
<code>CAP_FOWNER</code>	忽略文件许可的限制
<code>CAP_FSETID</code>	忽略 <code>setuid</code> 和 <code>setgid</code> 对文件施加的限制
<code>CAP_KILL</code>	发送信号时忽略 <code>ruid</code> 和 <code>euid</code>
<code>CAP_SETGID</code>	忽略与组相关的许可验证
<code>CAP_SETUID</code>	忽略与 <code>uid</code> 相关的许可验证
<code>CAP_SETCAP</code>	允许进程设置其权能

通过给 `capable()` 传递权能变量作为参数, 内核调用该函数检查是否设置了特定的权能。通常, 该函数检查在 `cap_effective` 集合中是否设置了该权能的相应位。若已设置, 则将 `current->flags` 置为 `PF_SUPERPRIV`, 意味着授予该权能, 同时返回 1; 否则, 返回 0。

与权能操作有关的系统调用包括 `capget()`、`capset()` 和 `prctl()`。前两个系统调用允许进程获取、设置其权能, 而 `prctl()` 则允许操作 `current->keep_capabilities`。

3.2.6 与进程限制相关的字段

进程通过硬件和调度程序来使用系统中的资源。下列字段可用于记录进程如何使用资源, 以

及该对进程进行何种限制。

rlim

rlim 字段是一个数组，该数组通过维护资源限制值对资源进行控制和记账。图 3-7 解释了 task_struct 中的 rlim 字段。

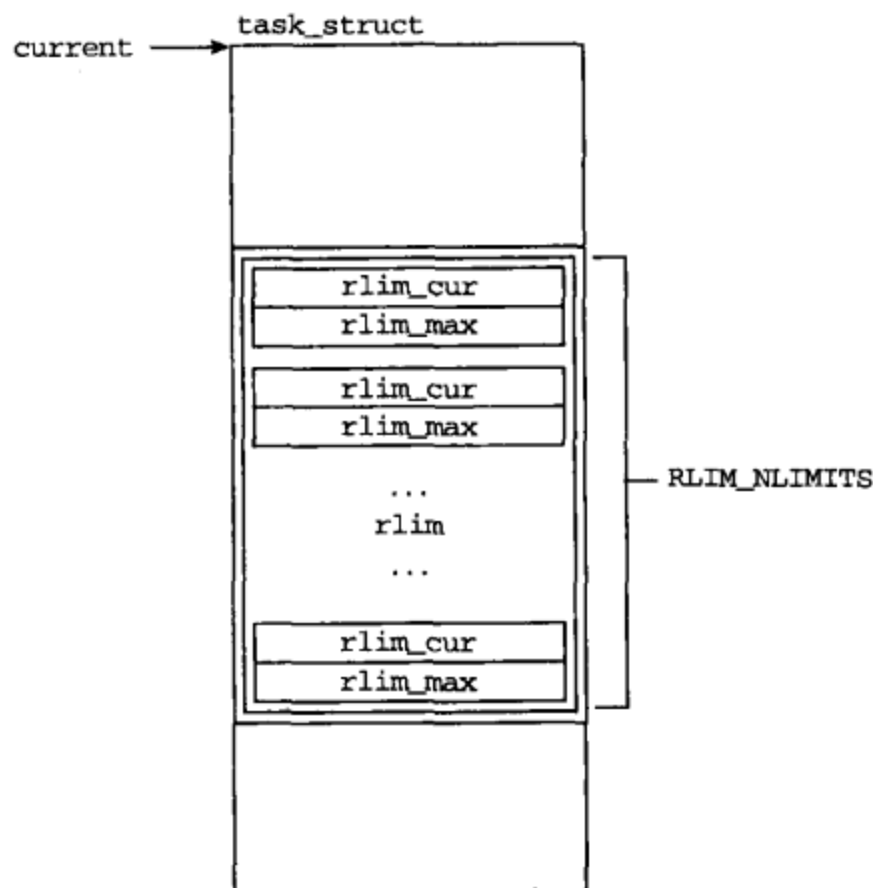


图 3-7 task_struct 中的资源限制

Linux 可识别是否需要限制允许一个进程使用的特定资源的数量。由于不同的进程可能使用的资源种类和数量都不同，因此有必要为每个进程存放这些基本信息。如果我们要保存这些信息，还有什么地方能比进程描述符更为合适呢？

`rlimit` 描述符 (`include/linux/resource.h`) 有 `rlim_cur` 和 `rlim_max` 这两个字段，分别表示资源的当前限制数目以及最大数目限制。这个限制的单位依该结构所指的资源种类的不同而有所不同。

```
-----
include/linux/resource.h
struct rlimit {
    unsigned long    rlim_cur;
    unsigned long    rlim_max;
};
-----
```

表 3-3 列出了一些资源，其限制数目在 `include/asm/resource.h` 中定义。x86 与 PPC 有相同的资源限制列表及其默认值。

表3-3 资源限制的值

RL名称	说 明	默认的rlim_cur	默认的rlim_max
RLIMIT_CPU	该进程使用CPU的时间量，单位为秒	RLIM_INFINITY	RLIM_INFINITY
RLIMIT_FSIZE	文件大小，单位为1 K	RLIM_INFINITY	RLIM_INFINITY
RLIMIT_DATA	堆大小，单位是字节	RLIM_INFINITY	RLIM_INFINITY
RLIMIT_STACK	栈大小，单位是字节	_STK_LIM	RLIM_INFINITY
RLIMIT_CORE	核心转储文件的大小	0	RLIM_INFINITY
RLIMIT_RSS	驻留集（实存）的最大值	RLIM_INFINITY	RLIM_INFINITY
RLIMIT_NPROC	该进程拥有的进程数	0	0
RLIMIT_NOFILE	进程一次最多可打开的文件数	INR_OPEN	INR_OPEN
RLIMIT_MEMLOCK	不可交换（可被锁定）内存的大小	RLIM_INFINITY	RLIM_INFINITY
RLIMIT_AS	进程地址空间的大小，单位是字节	RLIM_INFINITY	RLIM_INFINITY
RLIMIT_LOCKS	文件锁的数目	RLIM_INFINITY	RLIM_INFINITY

当上述的某个值被设置成 RLIM_INFINITY 时，该进程对这个资源的使用不再受限。

当前限制 rlim_cur 是一个软限制，调用 setrlimit() 可以对其进行修改，其最大限制由 rlim_max 来定义，任何没有相应特权的进程都不能超出这个最大值的限制。系统调用 getrlimit() 返回资源限制的值。setrlimit() 和 getrlimit() 都需要以资源的名称和一个指向 rlimit 类型结构的指针作为参数。

3.2.7 与文件系统及地址空间相关的字段

进程总是通过执行打开、关闭、读、写文件这样的任务与文件密切相关。task_struct 结构中有两个字段和“与文件以及文件系统相关”的数据有关：fs 和 files（详细内容请参见第 6 章），和地址空间相关的两个字段是 active_mm 和 mm（详细内容请参见第 4 章），图 3-8 所示为与文件系统和地址空间有关的 task_struct 的字段。

1. fs

fs 字段保存一个指向文件系统信息的指针。

2. files

files 字段保存一个指向进程的文件描述符表的指针。文件描述符也保存一个指针，它指向进程所打开的文件（更确切地说，是指向被打开的文件的描述符）。

3. mm

mm 指向与地址空间及内存管理相关的信息。

4. active_mm

active_mm 是指向最后访问的地址空间的指针。mm 和 active_mm 最初都是指向 mm_struct。

讨论进程描述符能使我们更好地理解进程在其整个生命周期中所涉及的数据及其类型。接下来将讨论在进程的整个生命期内会发生些什么。以下章节将阐释进程的不同生命阶段及其状态，并逐行分析一个实例，以解释在此期间内核中究竟发生了什么。

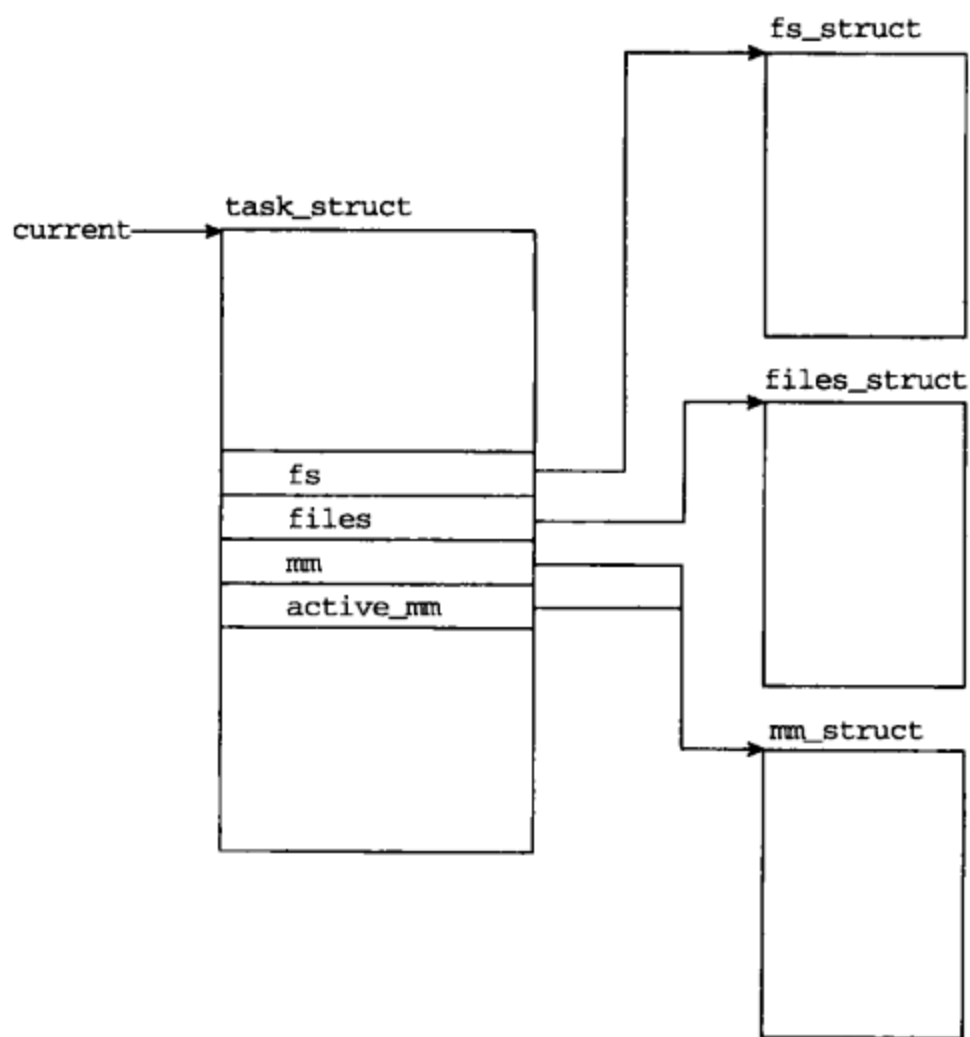


图 3-8 与文件系统和地址空间相关的字段

3.3 进程的创建：系统调用 `fork()`、`vfork` 和 `clone()`

当示例代码被编译成文件后（此处是指 ELF 可执行文件^①），就可以从命令行调用它。看看敲了回车键后会发生什么？前面已经说过，任何进程都是由其他进程创建的。操作系统通过系统调用 `fork()`、`vfork` 和 `clone()` 来完成进程的创建。

C 函数库提供了与这三个系统调用对应的函数。这些函数的原型是在文件 `<unistd.h>` 中声明的。图 3-9 解释了调用 `fork()` 函数的进程如何去执行系统调用 `sys_fork()`。该图描述了内核代码如何执行实际的进程创建。与此类似，`vfork()` 调用 `sys_fork()`，而 `clone()` 调用 `sys_clone()`。

这三个系统调用最终都调用了内核函数 `do_fork()`，该函数完成与进程创建有关的大部分工作。读者也许会奇怪创建进程为什么会有三个不同的函数？这三个函数在如何创建进程上有细微的差别，究竟选择哪一个函数都有其明确的理由。

在 shell 提示符下按下回车键时，shell 将通过调用 `fork()` 来创建新的进程，以便执行我们的程序。实际上，如果在 shell 提示符下输入 `ls` 命令并按回车，shell 在那一刻的伪代码类似于下面的语句：

^① ELF 可执行文件是 LINUX 支持的一种可执行文件格式。第 9 章将会讨论这种可执行文件格式。

```

if( (pid = fork()) == 0 )
    execve("foo");
else
    waitpid(pid);

```

现在来分析这几个函数，并且一直向下追踪到其系统调用。尽管我们的程序调用了 `fork()`，但调用 `vfork()` 或 `clone()` 也一样容易，这就是在本节对这三个函数都进行介绍的原因。第一个函数是 `fork()`，它调用了 `sys_fork()`，`sys_fork()` 又调用了 `do_fork()`。然后分析 `vfork()` 和 `clone()`，并跟踪其执行过程，直至调用 `do_fork()`。

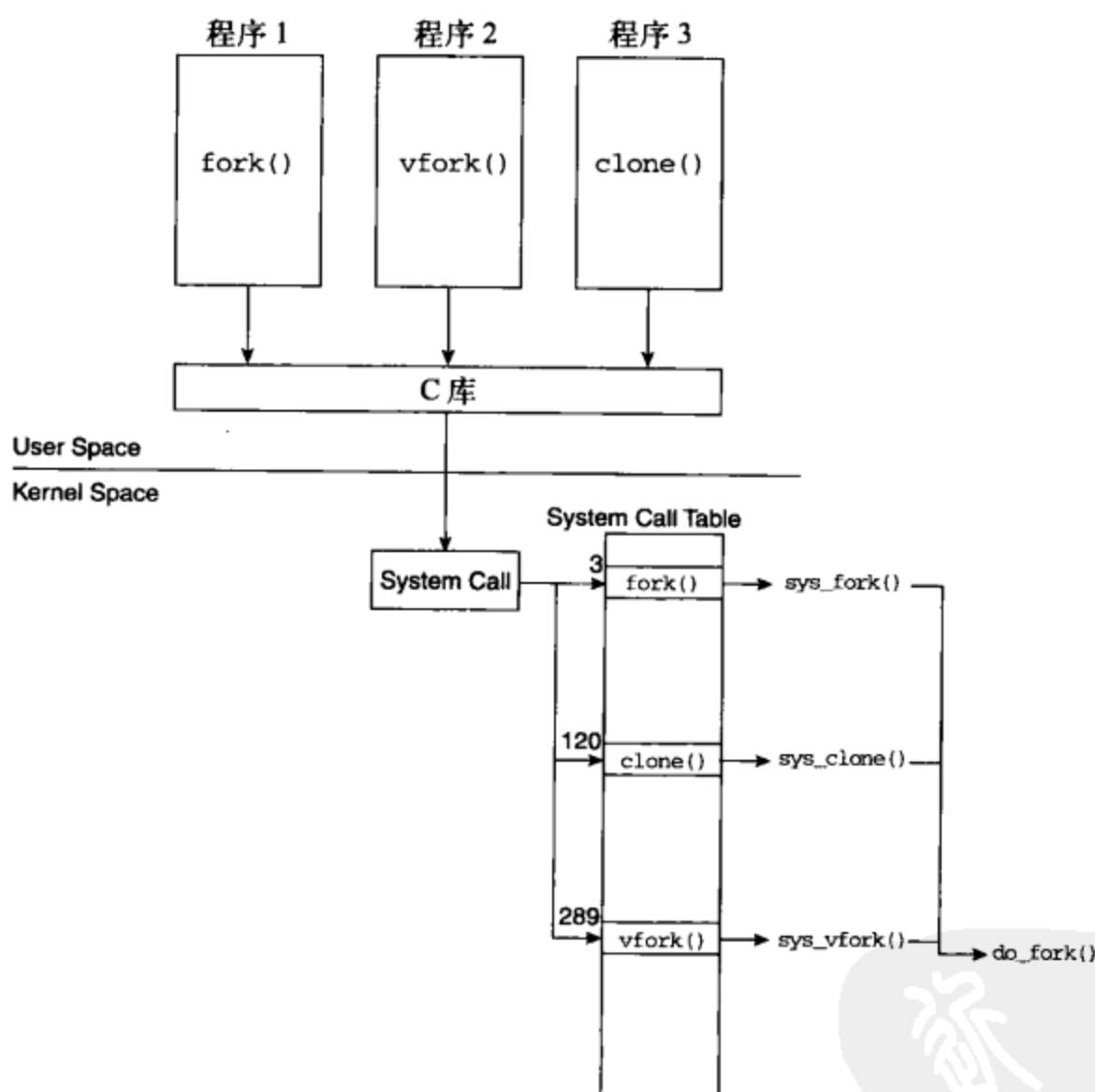


图 3-9 进程创建的系统调用

3.3.1 `fork()` 函数

`fork()` 函数返回了两次：一次是在父进程，一次是在子进程。如果在子进程中返回，将返回 0。如果在父进程中返回，将返回子进程的 PID。当 `fork()` 函数被调用时，该函数把包括该系统调用表索引在内的一些必要信息放入适当的寄存器，系统调用表中存放系统调用的指针由处理器来决定将这些信息放入哪个寄存器中。

关于这个问题，如果读者想继续深入了解，请查看 3.8.2 节中 `sys_fork()` 是如何被调用的。不过，不需理解一个新进程是如何创建的。

现在让我们来看看 `sys_fork()` 函数。这个函数仅仅是调用 `do_fork()` 函数。要注意的是，`sys_fork()` 函数是与体系结构相关的，因为其参数是由系统寄存器传来的。

```
-----
arch/i386/kernel/process.c
asmlinkage int sys_fork(struct pt_regs regs)
{
    return do_fork(SIGCHLD, regs.esp, &regs, 0, NULL, NULL);
}
-----

arch/ppc/kernel/process.c
int sys_fork(int p1, int p2, int p3, int p4, int p5, int p6,
             struct pt_regs *regs)
{
    CHECK_FULL_REGS(regs);
    return do_fork(SIGCHLD, regs->gpr[1], regs, 0, NULL, NULL);
}
-----
```

从上面的代码可以看出，这两种体系结构所接受的系统调用的参数也不同。`pt_regs` 存放了栈指针等信息。实际上，按照惯例，在 PPC 体系结构中，`gpr[1]` 存放了栈指针，而在 x86 体系结构中，`%esp`^① 存放了栈指针。

3.3.2 vfork() 函数

`vfork()` 函数和 `fork()` 函数类似，但 `vfork()` 的父进程一直阻塞，直到子进程调用 `exit()` 或 `exec()` 为止。

```
sys_vfork()
arch/i386/kernel/process.c
asmlinkage int sys_vfork(struct pt_regs regs)
{
    return do_fork(CLONE_VFORK | CLONE_VM | SIGCHLD, regs.ep, &regs, 0, NULL, NULL);
}
-----

arch/ppc/kernel/process.c
int sys_vfork(int p1, int p2, int p3, int p4, int p5, int p6,
             struct pt_regs *regs)
{
    CHECK_FULL_REGS(regs);
    return do_fork(CLONE_VFORK | CLONE_VM | SIGCHLD, regs->gpr[1],
                  regs, 0, NULL, NULL);
}
-----
```

在 `sys_vfork()` 中调用 `sys_fork()` 和直接调用 `sys_fork()` 的唯一区别就在于传给 `do_fork()` 的标志不相同。这些标志随后用来决定是否执行刚才所描述的阻塞的父进程的操作。

① 回忆一下，在“AT&T”格式的代码中，寄存器前要加一个前缀 %。

3.3.3 clone() 函数

库函数 clone() 不同于 fork() 和 vfork(), 它把一个指向函数的指针和该函数的参数作为自己的参数。由 do_fork() 创建的子进程刚一诞生就调用这个函数。

```

-----
sys_clone()
arch/i386/kernel/process.c
asmlinkage int sys_clone(struct pt_regs regs)
{
    unsigned long clone_flags;
    unsigned long newsp;
    int __user *parent_tidptr, *child_tidptr;

    clone_flags = regs.ebx;
    newsp = regs.ecx;
    parent_tidptr = (int __user *)regs.edx;
    child_tidptr = (int __user *)regs.edi;
    if (!newsp)
        newsp = regs.esp;
    return do_fork(clone_flags & ~CLONE_IDLETASK, newsp, &regs, 0,
parent_tidptr, child_tidptr);
}
-----

arch/ppc/kernel/process.c
int sys_clone(unsigned long clone_flags, unsigned long usp,
               int __user *parent_tidp, void __user *child_thread\
ptr,
               int __user *child_tidp, int p6,
               struct pt_regs *regs)
{
    CHECK_FULL_REGS(regs);
    if (usp == 0)
        usp = regs->gpr[1];    /* stack pointer for chi\
ld */
    return do_fork(clone_flags & ~CLONE_IDLETASK, usp, regs,\
0,
                  parent_tidp, child_tidp);
}
-----

```

如表 3-4 所示, fork()、vfork() 和 clone() 的唯一区别在于随后调用 do_fork() 时设置的标志不同。

表3-4 由fork()、vfork()和clone()传递给do_fork的标志

	fork()	vfork()	clone()
SIGCHLD	X	X	
CLONE_VFORK		X	
CLONE_VM		X	

最后，来看看 `do_fork()`，它才真正创建进程。到此为止，父进程调用了 `fork()`，然后它才能调用系统调用 `sys_fork()`；但仍然没有产生新的进程。程序 `foo` 仍然作为可执行文件存放在磁盘上，它还没有运行，也没有装入内存。

3.3.4 `do_fork()` 函数

我们逐行追踪 `do_fork()` 在内核中的执行，描述一下新进程创建背后的细节。

```
-----
kernel/fork.c
1167  long do_fork(unsigned long clone_flags,
1168             unsigned long stack_start,
1169             struct pt_regs *regs,
1170             unsigned long stack_size,
1171             int __user *parent_tidptr,
1172             int __user *child_tidptr)
1173  {
1174      struct task_struct *p;
1175      int trace = 0;
1176      long pid;
1177
1178      if (unlikely(current->ptrace)) {
1179          trace = fork_traceflag (clone_flags);
1180          if (trace)
1181              clone_flags |= CLONE_PTRACE;
1182      }
1183
1184      p = copy_process(clone_flags, stack_start, regs, stack_size, parent_tidptr,
child_tidptr);
-----
```

第 1178~1183 行：这几行代码首先检查父进程是否允许新进程被跟踪。追踪功能在处理进程的函数中普遍使用。本书只大概介绍 `ptrace` 功能。为了确定子进程是否被跟踪，`fork_traceflag()` 必须检查 `clone_flags` 的值。如果在 `clone_flags` 中设置了 `CLONE_VFORK` 标志，并且 `SIGCHLD` 信号没有被父进程捕获，或者如果当前进程也设置了 `PT_TRACE_FORK` 标志，那么子进程就会被跟踪，当然，如果设置了 `CLONE_UNTRACED` 或 `CLONE_IDLETASK` 标志，就不会被跟踪了。

第 1184 行：这一行创建了新进程并且复制了寄存器的值。`copy_process()` 函数完成新进程空间的创建和描述符字段定义的大部分工作，但是新进程并不立即运行。要了解 `copy_process()` 的更多细节，围绕调度程序去解释才会更清楚。3.6 节将详细讨论相关细节。

```
-----
kernel/fork.c
...
1189      pid = IS_ERR(p) ? PTR_ERR(p) : p->pid;
1190
1191      if (!IS_ERR(p)) {
1192          struct completion vfork;
1193
1194          if (clone_flags & CLONE_VFORK) {
```

```

1195         p->vfork_done = &vfork;
1196         init_completion(&vfork);
1197     }
1198
1199     if ((p->ptrace & PT_PTRACED) || (clone_flags & CLONE_STOPPED)) {
1200     ...
1203         sigaddset(&p->pending.signal, SIGSTOP);
1204         set_tsk_thread_flag(p, TIF_SIGPENDING);
1205     }
1206     ...

```

第 1189 行：这一行的代码是对指针错误的检查。如果发现指针出错，仅简单地返回指针错误。

第 1194~1197 行：这几行检查 do_fork() 是否被 vfork() 所调用。如果是，激活与 vfork() 相关的等待队列。

第 1199~1205 行：如果父进程被跟踪或者克隆操作被设置成 CLONE_STOPPED 标志，那么子进程在启动时将收到一个 SIGSTOP 信号，这样子进程就是以暂停状态启动的。

```

-----
kernel/fork.c
1207     if (!(clone_flags & CLONE_STOPPED)) {
1208     ...
1222         wake_up_forked_process(p);
1223     } else {
1224         int cpu = get_cpu();
1225
1226         p->state = TASK_STOPPED;
1227         if (!(clone_flags & CLONE_STOPPED))
1228             wake_up_forked_process(p); /* do this last */
1229         ++total_forks;
1230
1231         if (unlikely (trace)) {
1232             current->ptrace_message = pid;
1233             ptrace_notify ((trace << 8) | SIGTRAP);
1234         }
1235
1236         if (clone_flags & CLONE_VFORK) {
1237             wait_for_completion(&vfork);
1238             if (unlikely (current->ptrace & PT_TRACE_VFORK_DONE))
1239                 ptrace_notify ((PT_TRACE_EVENT_VFORK_DONE << 8) | SIGTRAP);
1240         } else
1241             ...
1248             set_need_resched();
1249     }
1250     return pid;
1251 }
-----

```

第 1226~1229 行：这几行将进程状态设置为 TASK_STOPPED。如果在 clone_flags 中未设置 CLONE_STOPPED 标志，那么就唤醒子进程；否则，让它等待一个唤醒信号。

第 1231~1234 行：如果在父进程上激活跟踪功能，则发送一个通知。

第 1236~1239 行：如果这是最初对 vfork() 的调用，那么就将父进程设置成阻塞状态，并发

送一个通知给跟踪者（如果父进程激活了跟踪功能的话）。这是通过把父进程放入等待队列中，并让它保持 `TASK_UNINTERRUPTIBLE` 状态直到子进程调用 `exit()` 或 `execve()` 来实现的。

第 1248 行：在当前进程（父进程）中设置 `need_resched`。这样就允许子进程先运行。

3.4 进程的生命周期

了解如何创建进程之后，有必要再了解一下在进程的整个生命周期中会发生什么。在此期间，进程总会发现自己处于各种状态。这些状态之间的转换既取决于进程所执行的操作，也取决于发送给进程的信号。本章给出的示例程序就处于 `TASK_INTERRUPTIBLE` 状态和 `TASK_RUNNING` 状态（其当前状态）。

进程所处的第一个状态是 `TASK_INTERRUPTIBLE`，这是在由 `do_fork()` 调用的 `copy_process()` 例程中创建进程时设置的。进程所处的第二个状态是 `TASK_RUNNING`，这是在退出 `do_fork()` 之前被设置的。这两个状态肯定会出现现在进程的生命周期中。接着会出现许多变化，这些变化决定进程处于何种状态。进程所处的最后一个状态是 `TASK_ZOMBIE`，这是在调用 `do_exit()` 期间被设置的。接下来看看进程的各种状态以及状态之间的转换方式，并解释进程是如何从一种状态转换到另一种状态的。

3.4.1 进程的状态

当进程正在运行时，意味着它的上下文已被装载到了 CPU 的寄存器和内存中，并且定义这个上下文的程序正在被执行。在某些特殊的时刻，有多种原因可能导致进程不能继续运行。这些原因有：进程正在等待还没有完成的输入；调度程序认为进程已经用完了分配给它的时间片，因此必须让其他进程运行。当进程能够运行却没有运行（比如重新调度）时被认为是就绪的，当等待输入时被认为是阻塞的。

图 3-10 显示了进程的抽象状态，每个抽象状态下列出了其对应的 Linux 可能的进程状态。表 3-5 给出了四种状态转换和引起转换的原因。表 3-6 将抽象状态和 Linux 内核中标识这些状态的值联系起来。

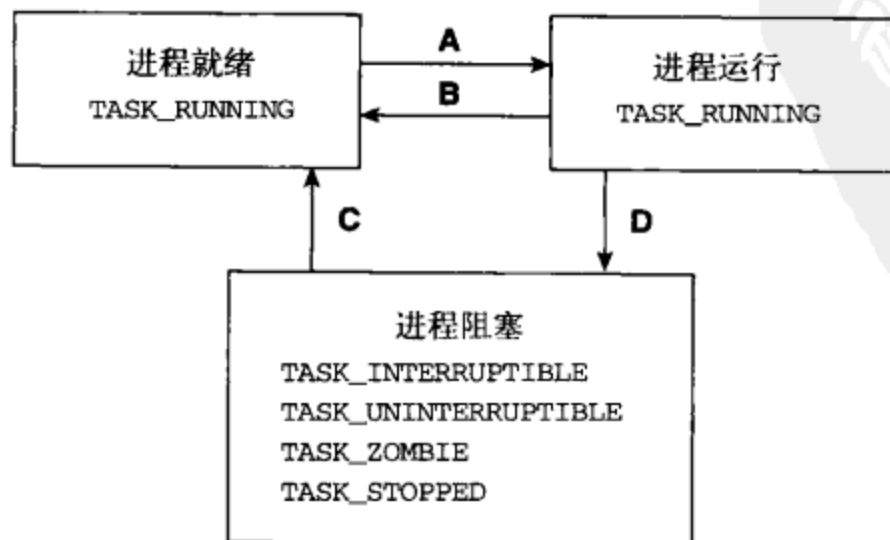


图 3-10 进程的状态转换

表3-5 进程的状态转换

转 换	原 因
就绪态→运行态 (A)	被调度程序选中
运行态→就绪态 (B)	时间片用完了 (被动的), 进程让出CPU (主动的)
阻塞态→就绪态 (C)	信号到来, 或等待的资源变为可用的
运行态→阻塞态 (D)	进程睡眠或等待事件

表3-6 Linux的标志和抽象进程状态间的关系

抽象状态	Linux的进程状态
就绪态	TASK_RUNNING
运行态	TASK_RUNNING
阻塞态	TASK_INTERRUPTIBLE TASK_UNINTERRUPTIBLE TASK_ZOMBIE TASK_STOPPED

注意 如果可以访问进程结构, `set_current_state()`就能直接设置其状态, 例如, 直接赋值设置 `current->state= TASK_INTERRUPTIBLE`。调用 `set_current_state(TASK_INTERRUPTIBLE)`可达到同样的效果。

3.4.2 进程状态的转换

现在来看看使进程从一种状态转换到另一种状态的各种事件。进程的抽象状态转换 (参阅表 3-5) 包括就绪态到运行态的转换、运行态到就绪态的转换、阻塞态到就绪态的转换和运行态到阻塞态的转换。每一种抽象状态转换至少对应一个 Linux 进程状态的转换。例如, 阻塞态到运行态的转换可以转化为 `TASK_INTERRUPTIBLE`、`TASK_UNINTERRUPTIBLE`、`TASK_ZOMBIE` 或 `TASK_STOPPED` 到 `TASK_RUNNING` 的转换。图 3-11 和表 3-7 详细描述了这些转换。

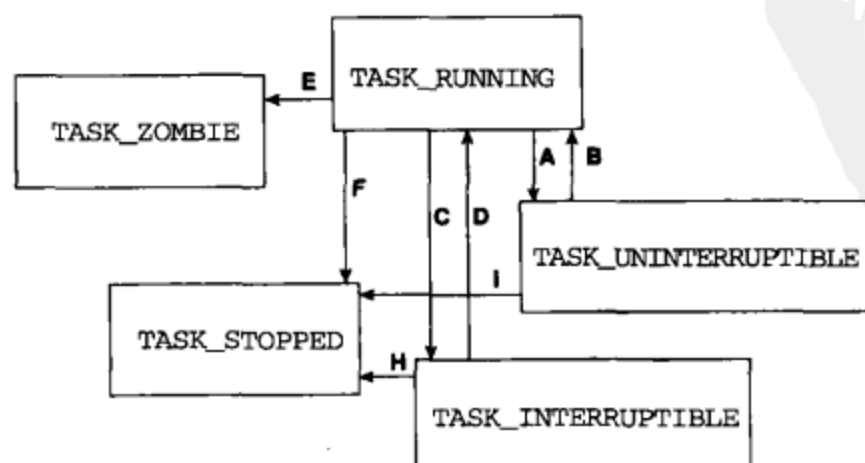


图 3-11 Linux 进程的状态转换

表3-7 Linux的进程状态转换

起始状态	结束状态	状态转换原因
TASK_RUNNING	TASK_UNINTERRUPTIBLE	进程进入等待队列
TASK_RUNNING	TASK_INTERRUPTIBLE	进程进入等待队列
TASK_RUNNING	TASK_STOPPED	进程接收到SIGSTOP信号., 或者进程被跟踪
TASK_RUNNING	TASK_ZOMBIE	进程被杀死, 但其父进程还没有调用sys_wait4()
TASK_INTERRUPTIBLE	TASK_STOPPED	收到信号时
TASK_UNINTERRUPTIBLE	TASK_STOPPED	被唤醒时
TASK_UNINTERRUPTIBLE	TASK_RUNNING	进程获得等待的资源时
TASK_INTERRUPTIBLE	TASK_RUNNING	进程获得等待的资源, 或收到信号后被设置为运行状态
TASK_RUNNING	TASK_RUNNING	由调度程序移入移出

接下来说明各种状态的转换, 并详细介绍普通进程转换类中的进程状态转换。

1. 从就绪态到运行态

“就绪态到运行态”的抽象状态转换并不对应实际的 Linux 进程状态转换, 因为进程的状态实际上没有发生变化 (进程仍然处于 TASK_RUNNING 状态)。然而, 进程确实进入了准备运行的队列 (或运行队列), 并真正由 CPU 执行。

● 从TASK_RUNNING到TASK_RUNNING

Linux 对当前正在使用 CPU 的进程并没有给出一个明确的状态, 即使进程已离开就绪队列, 并且它的上下文正在执行, 进程也一直保持着 TASK_RUNNING 状态。调度程序从运行队列中选择进程。在第 7 章将介绍调度程序如何选择下一个要运行的进程。

2. 从运行态到就绪态

在这种情况下, 即使进程本身发生了变化, 进程状态也不会改变。进程的抽象状态转换有助于读者理解究竟发生了什么。如前所述, 当进程从由 CPU 运行到进入运行队列时发生了状态转换, 经历了从运行到就绪的过程。

● 从TASK_RUNNING到TASK_RUNNING

由于 Linux 没有为其上下文由 CPU 执行的进程设置一个单独的状态, 因而此时进程不会经历一个明显的 Linux 状态转换, 还依然处于 TASK_RUNNING 状态。根据进程执行所花费时间的多少以及进程的优先级 (第 7 章将详细介绍), 调度程序决定何时将一个正在运行的进程切换下来放入运行队列。

3. 从运行态到阻塞态

当进程变为阻塞时, 它可能正处于如下某个状态: TASK_INTERRUPTIBLE、TASK_UNINTERRUPTIBLE、TASK_ZOMBIE 或 TASK_STOPPED, 如表 3-7 中所示, 接下来看看进程如何从 TASK_RUNNING 变成其中的每一个状态。

● TASK_RUNNING到TASK_INTERRUPTIBLE

这种状态通常是由 I/O 阻塞函数引起的, 因为这种函数必须等待一个事件或资源。对于一个

进程来说, 处于 TASK_INTERRUPTIBLE 状态意味着什么呢? 简单地说, 进程肯定不在运行队列中, 因为它还没有就绪。如果进程的资源变为可用 (时间或硬件), 或者如果一个信号到来, 处于 TASK_INTERRUPTIBLE 状态的进程就会被唤醒。最初的系统调用的完成依赖于中断处理程序的实现。在示例程序中, 子进程访问磁盘上的一个文件。磁盘驱动程序负责获知设备何时准备好供访问的数据。因此, 驱动程序中有一部分类似于下面这段代码:

```
while(1)
{
    if(resource_available)
        break();
    set_current_state(TASK_INTERRUPTIBLE);
    schedule();
}
set_current_state(TASK_RUNNING);
```

当示例中的进程执行 open() 调用时, 它将进入 TASK_INTERRUPTIBLE 状态。此处, 调用 schedule() 把它从 CPU 上换下来, 并选择运行队列的另外一个进程作为运行进程。在资源变为可用以后, 该进程终止循环并把其状态设置为 TASK_RUNNING, 这样该进程又被放回运行队列。进程在运行队列中等待, 直到调度程序认为轮到它执行为止。

以下是函数 interruptible_sleep_on(), 它能够将进程设置为 TASK_INTERRUPTIBLE 状态。

```
-----
kernel/sched.c
2504 void interruptible_sleep_on(wait_queue_head_t *q)
2505 {
2506     SLEEP_ON_VAR
2507
2508     current->state = TASK_INTERRUPTIBLE;
2509
2510     SLEEP_ON_HEAD
2511     schedule();
2512     SLEEP_ON_TAIL
2513 }
-----
```

SLEEP_ON_HEAD 宏和 SLEEP_ON_TAIL 宏分别完成向等待队列添加和删除一个进程 (参见 3.7 节) 的操作。SLEEP_ON_VAR 宏初始化进程的等待队列入口, 使得可以向等待队列添加进程。

● 从 TASK_RUNNING 到 TASK_UNINTERRUPTIBLE

除了当进程处于内核态时不理睬到来的信号外, TASK_UNINTERRUPTIBLE 状态与 TASK_INTERRUPTIBLE 状态类似。这个状态也是在调用 do_fork() 创建进程时, 为进程设置的默认状态。函数 sleep_on() 可以将进程设置为 TASK_UNINTERRUPTIBLE 状态。

```
-----
kernel/sched.c
2545 long fastcall __sched sleep_on(wait_queue_head_t *q)
2546 {
2547     SLEEP_ON_VAR
2548
2549     current->state = TASK_UNINTERRUPTIBLE;
```



```

2550
2551     SLEEP_ON_HEAD
2552     schedule();
2553     SLEEP_ON_TAIL
2554
2555     return timeout;
2556 }

```

该函数把进程放入等待队列，设置进程状态，并且调用调度程序。

- 从TASK_RUNNING到TASK_ZOMBIE

处于 TASK_ZOMBIE 状态的进程叫作**僵死进程**。每个进程在它的生命周期中都要经历这个状态。进程处于这种状态的时长依赖于它的父进程。为了更好地理解这一点，联想一下，在 UNIX 系统中，任何进程都可以通过调用 wait() 或 waitpid()（详细内容请参见 3.5.3 节）来获取子进程的退出状态。因此，即使子进程已经终止了，也还需要为父进程保留少量信息。仅仅因为父进程需要了解子进程的状态就让它一直活着，显然是很奢侈的事情；因此，进程有一种状态，除了进程描述符仍被保留外，其他资源都被释放并回收，这就是僵死状态。

设置 TASK_ZOMBIE 这个临时状态，是在调用 sys_exit()（详细内容参见第 3.5 节）时进行的。处于这种状态的进程不会再运行，只能转换到 TASK_STOPPED 状态。

如果一个进程处于僵死状态的时间过长，它的父进程将不会回收它。系统不能杀死僵死进程，因为它实际上就是死的。这意味着不存在杀死的僵死进程，只有等待被释放的进程描述符。

- 从TASK_RUNNING到TASK_STOPPED

这个转换会在两种情况下出现。第一种情况是正在被调试器或跟踪程序处理的进程；第二种情况是进程收到 SIGSTOP 或某种停止信号。

- 从TASK_UNINTERRUPTIBLE或者TASK_INTERRUPTIBLE到TASK_STOPPED

TASK_STOPPED 管理 SMP 系统中的进程或正在处理信号的进程。当进程收到一个唤醒信号时，或者内核明确需要进程不响应任何事情时（例如，如果进程被置为 TASK_INTERRUPTIBLE 状态，它是会响应其他事情的），进程就被设置为 TASK_STOPPED 状态。

与处于 TASK_ZOMBIE 状态的进程不同，处于 TASK_STOPPED 状态的进程仍然能接收 SIGKILL 信号。

4. 从阻塞态到就绪态

当进程得到等待的数据或硬件时，将从阻塞态转换到就绪态。这一状态转换对应着 Linux 的两种状态转换，分别是 TASK_INTERRUPTIBLE 到 TASK_RUNNING 和从 TASK_UNINTERRUPTIBLE 到 TASK_RUNNING。

3.5 进程的终止

进程可以显式、主动地终止，也可以隐式、主动地终止，或者被动终止。主动终止可通过两种途径来实现。

- (1) 从 main() 函数返回（隐式的）。
- (2) 调用 exit()（显式的）。

从主函数执行一个返回实际上转化为对 `exit()` 的调用。链接程序在这类情况下会引入对 `exit()` 的调用。

进程的被动终止可以通过三种途径来实现。

- (1) 进程可能收到自己不能处理的一个信号。
- (2) 当进程在内核态执行时可能产生一个异常。
- (3) 进程可能收到了 `SIGABRT` 信号或其他终止信号。

对进程的终止如何处理依赖于父进程是否消亡。一个进程可能：

- 先于父进程终止；
- 在父进程之后终止。

在第一种情况下，子进程被变成一个僵死进程，直到父进程调用 `wait()` 或 `waitpid()`。在第二种情况下，`init()` 进程将成为子进程的新父进程。当任何进程终止的时候，内核都要查看一遍所有活着的进程，核实将要终止的进程是否是某个活着进程的父进程，如果是，内核就把其子进程的父进程 `PID` 修改为 1。

接下来再来看一下示例程序，并且跟踪它直到消亡。该进程显式地调用了 `exit(0)`。（注意，调用 `_exit()`、`return(0)` 或者从 `main` 函数的结束处退出也是一样的。）C 库函数 `exit()` 接着调用 `sys_exit()` 系统调用。接下来分析下面的代码，看看从这里开始对进程做了些什么。

首先，来看看终止进程的函数。如前所述，进程 `foo` 调用了 `exit()`，`exit()` 接着调用了第一个函数——`sys_exit()`。我们将追踪 `sys_exit()` 的调用，并且一直深入到 `do_exit()` 的内部。

3.5.1 `sys_exit()` 函数

```
-----
kernel/exit.c
asmlinkage long sys_exit(int error_code)
{
    do_exit((error_code&0xff)<<8);
}
-----
```

`sys_exit()` 不随体系结构而变化，它的工作也相当简单，仅仅是把退出码转换成内核要求的格式并调用 `do_exit()`。

3.5.2 `do_exit()` 函数

```
-----
kernel/exit.c
707 NORET_TYPE void do_exit(long code)
708 {
709     struct task_struct *tsk = current;
710
711     if (unlikely(in_interrupt()))
712         panic("Aieee, killing interrupt handler!");
713     if (unlikely(!tsk->pid))
714         panic("Attempted to kill the idle task!");
715     if (unlikely(tsk->pid == 1))
```

```

716    panic("Attempted to kill init!");
717    if (tsk->io_context)
718        exit_io_context();
719    tsk->flags |= PF_EXITING;
720    del_timer_sync(&tsk->real_timer);
721
722    if (unlikely(in_atomic()))
723        printk(KERN_INFO "note: %s[%d] exited with preempt_count %d\n",
724                current->comm, current->pid,
725                preempt_count());
-----

```

第 707 行：参数 code 包含了进程返回给父进程的退出代码。

第 711~716 行：排除一些无效但可能发生的情况，包括以下两种情况。

(1) 确定退出进程没有处于中断处理中。

(2) 确保退出进程不是 idle 进程 (PID=0) 或 init 进程 (PID=1)。注意，init 进程只有在系统关闭时才能被杀死。

第 719 行：在此，将进程的进程结构的 flag 字段设置为 PF_EXITING。这表明进程正在关闭。例如，为一个给定进程创建间隔定时器时要使用该标志。检查进程的标志是否设置有助于防止多余的处理。

```

-----
kernel/exit.c
...
727    profile_exit_task(tsk);
728
729    if (unlikely(current->ptrace & PT_TRACE_EXIT)) {
730        current->ptrace_message = code;
731        ptrace_notify((PTRACE_EVENT_EXIT << 8) | SIGTRAP);
732    }
733
734    acct_process(code);
735    __exit_mm(tsk);
736
737    exit_sem(tsk);
738    __exit_files(tsk);
739    __exit_fs(tsk);
740    exit_namespace(tsk);
741    exit_thread();
...
-----

```

第 729~732 行：如果进程正在被追踪并且已经设置了 PT_TRACE_EXIT 标志，那么就传递退出代码并通知父进程。

第 735~742 行：这几行清理和回收进程已经使用过并且以后不再使用的资源。__exit_mm() 释放分配给进程的内存，并释放与进程相关的 mm_struct。exit_sem() 把进程从所有 IPC 信号量中分离出来。__exit_files() 释放分配给进程的所有文件，减小文件描述符的计数。__exit_fs() 释放所有的文件系统数据。

```

-----
kernel/exit.c
...
744  if (tsk->leader)
745      disassociate_ctty(1);
746
747  module_put(tsk->thread_info->exec_domain->module);
748  if (tsk->binfmt)
749      module_put(tsk->binfmt->module);
...
-----

```

第 744~745 行：如果进程是一个会话首进程，它极可能拥有一个控制终端或 `tty`。这个函数把会话首进程从它的控制终端 `tty` 中分离出来。

第 747~749 行：这几行减少模块的引用数。

```

-----
kernel/exit.c
...
751  tsk->exit_code = code;
752  exit_notify(tsk);
753
754  if (tsk->exit_signal == -1 && tsk->ptrace == 0)
755      release_task(tsk);
756
757  schedule();
758  BUG();
759  /* Avoid "noreturn function does return". */
760  for (;;) ;
761  }
...
-----

```

第 751 行：在 `task_struct` 的 `exit_code` 字段中设置退出代码。

第 752 行：给父进程发送 `SIGCHLD` 信号，并设置进程状态为 `TASK_ZOMBIE`。`exit_notify()` 向垂死进程的亲属通报进程的消亡。此时，该进程的父进程将接到退出代码的通知，而其子进程则将自己的父进程设置为 `init` 进程。唯一例外的是，如果该进程所在的组中还有其他进程存在，那么现存的这个进程就作为其子进程的代理父进程。

第 754 行：如果 `exit_signal` 是 `-1`（表明有一个错误），并且进程没有被跟踪，那么内核将调用调度程序去释放这个进程的描述符并回收它的时间片。

第 757 行：将处理器让给新进程。在第 7 章中将看到，调用 `schedule()` 不会有返回值。此后的所有代码捕获不太可能发生的情况，或者消除编译器的警告。

3.5.3 通知父进程和 `sys_wait4()`

当一个进程终止时，要通知它的父进程。在这之前，进程处于僵死状态，在此状态中，除了进程描述符仍然保留着以外，它的所有资源都已归还内核。当子进程终止时，父进程（例如 `Bash shell`）收到内核发送给它的 `SIGCHLD` 信号。本例中，当 `shell` 想要收到通知时就调用 `wait()`。父

进程可以通过不执行中断处理程序来忽略信号，也可以改为选择在任何时刻调用 `wait()` (或 `waitpid()`)。

`wait` 函数族扮演以下两种常见的角色。

❑ 殡仪业者。获知进程消亡的消息。

❑ 掘墓人。消除进程的所有痕迹。

父进程可以选择调用 `wait` 函数族中的 4 个函数之一：

❑ `pid_t wait(int *status)`

❑ `pid_t waitpid(pid_t pid, int *status, int options)`

❑ `pid_t wait3(int *status, int options, struct rusage *rusage)`

❑ `pid_t wait4(pid_t pid, int *status, int options, struct rusage *rusage)`

每个函数又依次调用 `sys_wait4()`，大部分的通知会在 `sys_wait4()` 中产生。

如果进程调用 `wait` 函数族中的某一函数，那么它会被阻塞，直到它的某个子进程终止，或者如果子进程已经终止（或其父进程已经没有其他子进程），该进程就立即返回。`sys_wait4()` 函数显示内核如何管理这个通知：

```
-----
kernel/exit.c
1031  asmlinkage long sys_wait4(pid_t pid,unsigned int * stat_addr,
int options, struct rusage * ru)
1032  {
1033      DECLARE_WAITQUEUE(wait, current);
1034      struct task_struct *tsk;
1035      int flag, retval;
1036
1037      if (options & ~(WNOHANG|WUNTRACED|__WNOTHREAD|__WCLONE|__WALL))
1038          return -EINVAL;
1039
1040      add_wait_queue(&current->wait_chldexit,&wait);
1041      repeat:
1042          flag = 0;
1043          current->state = TASK_INTERRUPTIBLE;
1044          read_lock(&tasklist_lock);
1045      ...
-----
```

第 1031 行：这些参数包括目标进程的 PID、存放子进程退出状态的地址、传给 `sys_wait4()` 的标志以及存放子进程资源使用信息的地址。

第 1033 和 1040 行：声明一个等待队列，并把进程加入该队列。（详细内容参见 3.7 节。）

第 1037~1038 行：这两行代码主要检查错误条件。如果传给 `sys_wait4()` 的参数 `options` 是无效的，函数将返回一个错误码。在本例中，返回错误码 `EINVAL`。

第 1042 行：`flag` 变量的初始值被设置为 0。一旦发现参数 `pid` 和调用进程的某个子进程的 `pid` 匹配，就要修改这个变量。

第 1043 行：将调用进程设置为阻塞状态，进程的状态从 `TASK_RUNNING` 变为 `TASK_INTERRUPTIBLE`。

```

-----
kernel/exit.c
...
1045     tsk = current;
1046     do {
1047         struct task_struct *p;
1048         struct list_head *_p;
1049         int ret;
1050
1051         list_for_each(_p, &tsk->children) {
1052             p = list_entry(_p, struct task_struct, sibling);
1053
1054             ret = eligible_child(pid, options, p);
1055             if (!ret)
1056                 continue;
1057             flag = 1;
1058             switch (p->state) {
1059                 case TASK_STOPPED:
1060                     if (!(options & WUNTRACED) &&
1061                         !(p->ptrace & PT_PTRACED))
1062                         continue;
1063                     retval = wait_task_stopped(p, ret == 2,
1064                         stat_addr, ru);
1065                     if (retval != 0) /* He released the lock. */
1066                         goto end_wait4;
1067                     break;
1068                 case TASK_ZOMBIE:
1069
1070                     if (ret == 2)
1071                         continue;
1072                     retval = wait_task_zombie(p, stat_addr, ru);
1073                     if (retval != 0) /* He released the lock. */
1074                         goto end_wait4;
1075                     break;
1076             }
1077         }
1078     }
1079
1080     ...
1091     tsk = next_thread(tsk);
1092     if (tsk->signal != current->signal)
1093         BUG();
1094 } while (tsk != current);
...
-----

```

第 1046 行和 1094 行：当 do while 循环检测到自己还是当前进程时只执行一次循环，当检测到是其他进程时便继续循环。

第 1051 行：对进程的子进程列表中的每个进程重复操作。请注意，是父进程在等待子进程的退出。父进程当前处于 TASK_INTERRUPTIBLE 状态，正在逐个扫描其子进程。

第 1054 行：确定被传递的 pid 参数是否不合理。

第 1058~1079 行：检查进程的每个子进程的状态。仅当其子进程停止或僵死时才进行这种处理，如果进程正在睡眠、就绪或运行（其余状态），则什么也不做。如果子进程是 TASK_STOPPED 状态，并且使用了 UNTRACED 选项（这意味着进程由于跟踪而未被停止），则核实是否已经报告了子进程的状态，并且返回子进程的信息。如果子进程是 TASK_ZOMBIE 状态，则撤销它。

```

-----
kernel/exit.c
...
1106  retval = -ECHILD;
1107  end_wait4:
1108  current->state = TASK_RUNNING;
1109  remove_wait_queue(&current->wait_chldexit,&wait);
1110  return retval;
1111  }
-----

```

第 1106 行：如果执行到了这里，说明参数所指定的 PID 不是所调用进程的子进程。ECHILD 错误用于通报这种事件。

第 1107~1111 行：至此，子进程列表处理完毕，所有需要撤销的子进程都已被撤销。父进程的阻塞被解除，它的状态又被设置为 TASK_RUNNING。最后，等待队列也被删除。

至此，读者应该对进程在它的生命周期中所经历的各种状态、完成状态转换的内核函数，以及内核用于记录所有这些信息的数据结构都很熟悉了。接下来再看一看调度程序如何控制和管理进程以达到多线程系统的效果。当然也会看到进程状态转换的更多细节。

3.6 了解进程的动态：调度程序的基本构架

迄今为止，我们了解了以进程为中心的状态和状态转换，但还没有谈及如何管理进程状态的转换，也没有讨论执行进程的运行和停止操作的内核基本框架结构。调度程序将处理所有这些细节。完成了对进程生命周期的探索后，现在来介绍调度程序的基本知识，以及进程创建期间调度程序如何与函数 `do_fork()` 交互。

3.6.1 基本结构

调度程序操作的对象是一个称为运行队列的结构。系统中的每个 CPU 都有一个运行队列。运行队列中的核心数据结构是两个按优先级排序的数组。其中一个包含了活跃进程，另外一个包含了到期进程。通常，一个活跃进程运行一段固定的时间（时间片长度或时间片），然后被插入到期数组去等待更多的 CPU 时间。当活跃数组为空时，调度程序通过交换活跃数组和到期数组的指针来交换两个数组。然后，调度程序开始执行新活跃数组中的进程。

图 3-12 说明了运行队列中的优先级数组。优先级数组结构的定义如下：

```

-----
kernel/sched.c
192  struct prio_array {
193      int nr_active;
194      unsigned long bitmap[BITMAP_SIZE];
195      struct list_head queue[MAX_PRIO];
196  };
-----

```

`prio_array` 结构的字段如下所示。

□ **nr_active**。计数器，记录优先级数组中的进程数。

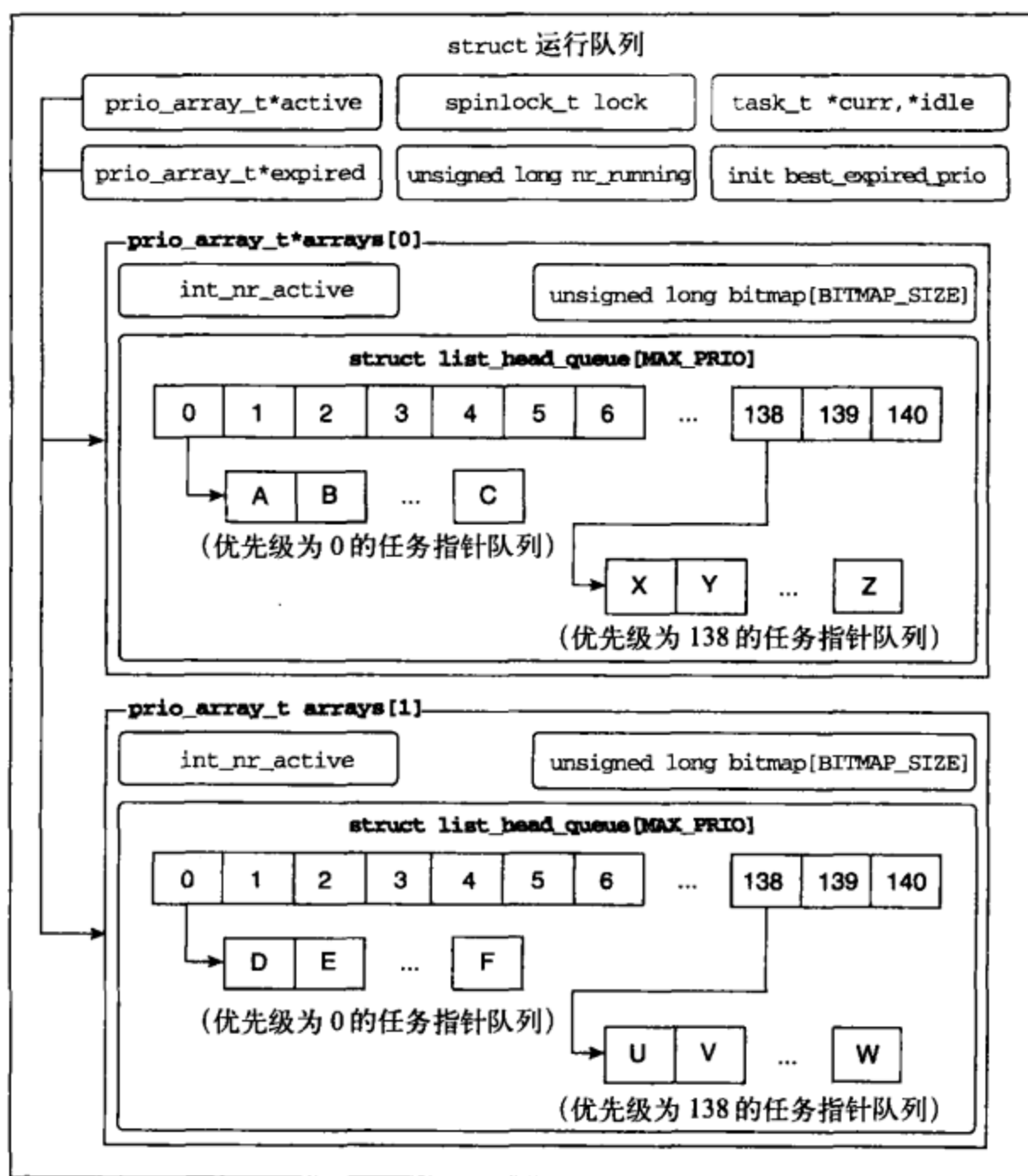


图 3-12 运行队列中的优先级数组

- **bitmap**。记录数组中的优先级。bitmap 的实际长度取决于系统中无符号长整型的大小。它始终足够存放 MAX_PRIO 个位，但也可能会更长。
- **queue**。存储进程链表的数组。每个链表存放特定优先级的进程，因此，queue[0]就是存放所有优先级为 0 的进程的链表，queue[1]就是存放所有优先级为 1 的进程的链表，等等。

在基本理解了运行队列是如何组织的之后，再通过单 CPU 系统上的调度程序来着手理解进程。

3.6.2 从等待中醒来或者激活

回想一下，当进程调用 `fork()` 时，就创建了新的进程。如前所述，调用 `fork()` 的进程叫做父进程，新的进程叫做子进程。新创建的进程需要被调度才能访问 CPU，这是通过 `do_fork()` 函数来完成的。

`do_fork()`中有两行用于处理调度程序的重要代码，这两行代码同时还与唤醒进程有关。在 `linux/kernel/fork.c` 的第 1 184 行调用了函数 `copy_process()`，该函数将调用函数 `sched_fork()`，以便把进程初始化为即将插入调度程序运行队列的状态。在 `linux/kernel/fork.c` 的第 1 222 行调用了函数 `wake_up_forked_process()`，它用于获取已初始化的进程，并把它真正插入运行队列。考虑到新进程可能被杀死，初始化操作和插入操作被分开进行，否则的话，进程可能还没有被调度就已经被终止了。只有创建成功、初始化成功且没有挂起的信号的进程才能被调度。

`sched_fork()`：调度程序初始化新创建的进程

调度程序需要初始化新创建的进程，`sched_fork()`函数就完成这种基础结构的设置。

```
-----
kernel/sched.c
719 void sched_fork(task_t *p)
720 {
721     /*
722      * We mark the process as running here, but have not actually
723      * inserted it onto the runqueue yet. This guarantees that
724      * nobody will actually run it, and a signal or other external
725      * event cannot wake it up and insert it on the runqueue either.
726      */
727     p->state = TASK_RUNNING;
728     INIT_LIST_HEAD(&p->run_list);
729     p->array = NULL;
730     spin_lock_init(&p->switch_lock);
-----
```

第 727 行：在 `do_fork()` 和 `copy_process()` 对进程是否正确创建进行验证之前，就把进程标记为运行状态（这是通过把进程结构的 `State` 属性设置为 `TASK_RUNNING` 来实现的），以确保没有其他事件能把该进程插入运行队列并运行它。当这些验证都通过以后，`do_fork()`才调用 `wake_up_forked_process()` 把进程真正添加到运行队列中。

第 728~730 行：初始化进程的 `run_list` 字段。当进程被激活时，它的 `run_list` 字段被链接到运行队列中某个优先级数组的队列结构中。把进程的 `array` 域置为 `NULL`，表示进程不在运行队列的任何一个优先级数组中。`sched_fork()`的下一部分代码（第 731~739 行）处理内核的抢占（关于抢占的详细内容参见第 7 章）。

```
-----
kernel/sched.c
740     /*
741      * Share the timeslice between parent and child, thus the
742      * total amount of pending timeslices in the system doesn't change,
743      * resulting in more scheduling fairness.
744      */
745     local_irq_disable();
746     p->time_slice = (current->time_slice + 1) >> 1;
747     /*
748      * The remainder of the first timeslice might be recovered by
749      * the parent if the child exits early enough.
750      */
751     p->first_time_slice = 1;
-----
```

```

752  current->time_slice >>= 1;
753  p->timestamp = sched_clock();
754  if (!current->time_slice) {
755      /*
756       * This case is rare, it happens when the parent has only
757       * a single jiffy left from its timeslice. Taking the
758       * runqueue lock is not a problem.
759       */
760      current->time_slice = 1;
761      preempt_disable();
762      scheduler_tick(0, 0);
763      local_irq_enable();
764      preempt_enable();
765  } else
766      local_irq_enable();
767 }

```

第 740~753 行：在禁用本地中断之后，用移位运算符给子进程划分父进程的一部分时间片。新进程的第一个时间片被设置为 1，因为它还未被运行，其时间戳被初始化成以纳秒为单位的当前时间。

第 754~767 行：如果父进程的时间片是 1，这种划分将导致父进程没有剩余的时间运行。因为父进程是调度程序的当前进程，因此需要调度程序选择一个新的进程来运行。这可以通过调用 `scheduler_tick()`（见第 762 行）来完成。为了确保调度程序不受干扰地选择一个新进程，应该禁止内核被抢占。调度操作完成以后，则启用抢占并恢复本地中断。

此处，新创建的进程初始化其与调度程序相关的变量，并让其初始时间片为父进程剩余时间片的一半。内核强迫进程牺牲分配给它的一部分 CPU 时间，并把这些时间分给其子进程来防止进程占有大块的处理时间。如果进程被给予一个固定不变的时间片，恶意进程可能会生出许多子进程，从而快速而贪婪地占有 CPU。

进程被成功地初始化并且初始化被验证有效以后，`do_fork()` 就调用 `wake_up_forked_process()`：

```

-----
kernel/sched.c
922 /*
923  * wake_up_forked_process - wake up a freshly forked process.
924  *
925  * This function will do some initial scheduler statistics housekeeping
926  * that must be done for every newly created process.
927  */
928 void fastcall wake_up_forked_process(task_t * p)
929 {
930     unsigned long flags;
931     runqueue_t *rq = task_rq_lock(current, &flags);
932
933     BUG_ON(p->state != TASK_RUNNING);
934
935     /*
936     * We decrease the sleep average of forking parents

```

```

937  * and children as well, to keep max-interactive tasks
938  * from forking tasks that are max-interactive.
939  */
940  current->sleep_avg = JIFFIES_TO_NS(CURRENT_BONUS(current) *
941    PARENT_PENALTY / 100 * MAX_SLEEP_AVG / MAX_BONUS);
942
943  p->sleep_avg = JIFFIES_TO_NS(CURRENT_BONUS(p) *
944    CHILD_PENALTY / 100 * MAX_SLEEP_AVG / MAX_BONUS);
945
946  p->interactive_credit = 0;
947
948  p->prio = effective_prio(p);
949  set_task_cpu(p, smp_processor_id());
950
951  if (unlikely(!current->array))
952    __activate_task(p, rq);
953  else {
954    p->prio = current->prio;
955    list_add_tail(&p->run_list, &current->run_list);
956    p->array = current->array;
957    p->array->nr_active++;
958    rq->nr_running++;
959  }
960  task_rq_unlock(rq, &flags);
961  }

```

第930~934行：调度程序做的第一件事就是锁住运行队列结构。对运行队列的任何更改必须在上锁情况下进行。如果进程没有被标记为 TASK_RUNNING（通过 sched_fork() 初始化后应该为 TASK_RUNNING，参见前面列出的 kernel/sched.c 中的第727行），那么就抛出一个 bug 通知。

第940~947行：调度程序计算父进程和子进程的睡眠平均值。睡眠平均值是进程睡眠所花时间和进程运行所花时间的比。它随着进程睡眠时间数而增加，随着进程运行时每个定时器的节拍而减少。交互式进程或者 I/O 密集型（I/O bound）进程在等待输入上花费了大部分时间，它们的睡眠平均值通常很高。非交互式进程或者 CPU 密集型（CPU-bound）进程在使用 CPU（而不是等待 I/O）上花费了大部分时间，这些进程的睡眠平均值很低。因为用户想看到他们的输入结果，例如敲击键盘或移动鼠标，因此交互式进程被赋予了比非交互式进程更多的调度优势。具体而言，在交互式进程的时间片到期以后，调度程序把它重新插入到活跃的优先级数组中。为防止交互式进程创建一个非交互的子进程，从而占有一个不相称的 CPU 份额，这些公式用来降低父进程和子进程的睡眠平均值。如果新创建的进程是交互式的，它立刻去睡眠足够长的时间，重新获得本已失去的所有调度优势。

第948行：函数 effective_prio() 修改进程的静态优先级。它返回一个 100~139（MAX_RT_PRIO 到 MAX__PRIO-1）的优先级。基于进程以前使用 CPU 的情况以及用于睡眠的时间，进程的静态优先级可以增加或减少一个不大于 5 的数，但总保持在刚才提到的范围内。我们已经从命令行开始讨论了进程的 nice 值，它可以在 +19~20 的范围内变化（最低优先级至最高优先级）。nice 优先级 0 对应静态优先级的 120。

第 949 行：把进程的 CPU 属性设置成当前 CPU。

第 951~960 行：总的来看，这段代码是新进程（或者说子进程）从当前进程——它的父进程复制调度信息，然后把自己插入运行队列的适当位置。至此，完成了对运行队列的修改，因此将它解锁。下面的段落和图 3-13 将更详细地讨论这个进程。

指针 `array` 指向运行队列中的优先级数组。如果当前进程没有指向一个优先级数组，那就意味着当前进程已经完成或者正在睡眠。如果是那样的话，当前进程的 `runlist` 字段就不在运行队列的优先级数组的队列中，这意味着 `list_add_tail()`（第 955 行）操作会失败。因此改为利用 `__activate_task()` 来插入新创建的进程，这个函数就把新进程添加到运行队列中而无需涉及它的父进程。

通常，当前进程正在运行队列中等待 CPU 时间时，进程被添加到优先级数组中处于 `p->prio` 位置的队列。添加了新进程的数组其进程计数器 `nr_active` 增加 1，运行队列也让它的进程计数器 `nr_running` 增加 1。最后，将运行队列解锁。

当前进程不指向运行队列中的优先级数组时，这对于观察调度程序如何管理运行队列和优先级数组的属性是很有用处的。

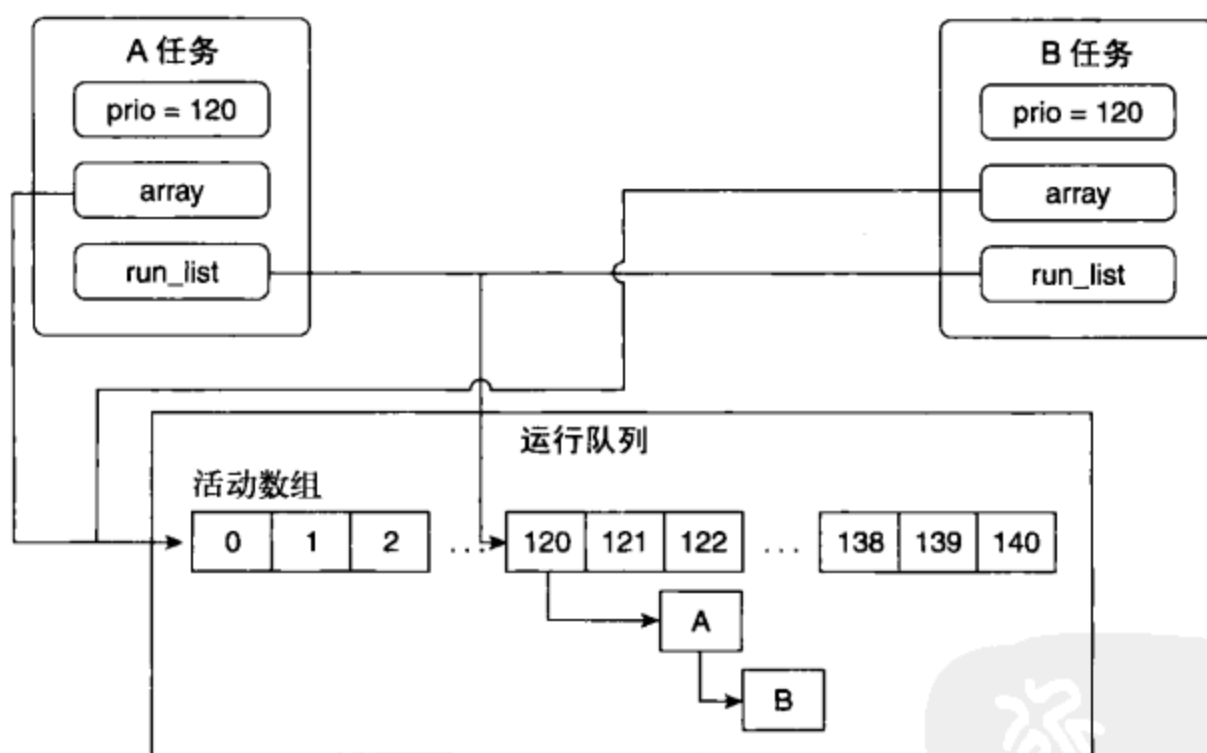


图 3-13 运行队列的插入

```
-----
kernel/sched.c
366 static inline void __activate_task(task_t *p, runqueue_t *rq)
367 {
368     enqueue_task(p, rq->active);
369     rq->nr_running++;
370 }
-----
```

`__activate_task()` 把给定进程 `p` 放到运行队列 `rq` 中的活跃优先级数组上，并且把运行队列的 `nr_running` 字段增加 1，`nr_running` 是一个表示运行队列上进程总数的计数器。

```

-----
kernel/sched.c
311 static void enqueue_task(struct task_struct *p, prio_array_t *array)
312 {
313     list_add_tail(&p->run_list, array->queue + p->prio);
314     __set_bit(p->prio, array->bitmap);
315     array->nr_active++;
316     p->array = array;
317 }
-----

```

第 311~312 行：当初始化优先级数组时，`enqueue_task()` 获得进程 `p`，并把它放到优先级数组 `array` 上。

第 313 行：进程的 `run_list` 被添加到优先级数组中位于 `p->prio` 处队列的队尾。

第 314 行：设置优先级为 `p->prio` 的优先级数组位图，当调度程序运行时，就能够看到有一个进程以优先级 `p->prio` 运行。

第 315 行：增加优先级数组的进程计数器，以反映加入了新进程。

第 316 行：设置进程的数组指针 `p->array`，使之指向刚添加了进程的优先级数组。

简而言之，即使由于整个调度程序中很多类似的名字使得代码容易让人混淆，添加一个新创建进程的操作还是相当简单的。进程被放置在运行队列优先级数组中某个链表的末尾，该链表在数组中的位置是由进程的优先级来确定的。然后，进程在它的数据结构中记录优先级数组的位置和它所在链表的位置。

3.7 等待队列

前面已经讨论了 `TASK_RUNNING` 状态和 `TASK_INTERRUPTIBLE` 或 `TASK_UNINTERRUPTIBLE` 状态之间的转换。现在来看看涉及这一状态转换的另一个结构。当进程等待一个外部事件发生时，就把它从运行队列删除并放到等待队列上。等待队列是一个 `wait_queue_t` 结构的双向链表。设置 `wait_queue_t` 结构是为了保存记录等待进程所需的所有信息。等待一个特定外部事件的所有进程被放到同一个等待队列中。当某个等待队列上的进程被唤醒时，该进程核实它所等待的条件，然后或者继续睡眠，或者从等待队列删除自己并把自己设置回 `TASK_RUNNING`。回想一下，当父进程想要获知它所创建的子进程的状态时，`sys_wait4()` 系统调用使用了等待队列。注意，等待外部事件的进程（因此不再处于运行队列^①中）可能处于 `TASK_INTERRUPTIBLE` 状态，也可能处于 `TASK_UNINTERRUPTIBLE` 状态。

等待队列是 `wait_queue_t` 结构的双向链表，`wait_queue_t` 结构中有指向阻塞进程 `task` 结构的指针。每个链表以 `wait_queue_head_t` 结构打头，该结构标记链表的头部，并存放 `wait_queue_t` 链表的自旋锁，自旋锁可以防止 `wait_queue_t` 的额外竞争条件。图 3-14 说明了等待队列是如何实现的。现在来看看 `wait_queue_t` 和 `wait_queue_head_t` 结构。

```

-----
include/linux/wait.h

```

① 进程睡眠以后，它就被从运行队列删除，从而为另外一个进程让出 CPU 的控制权。


```

19 typedef struct __wait_queue wait_queue_t;
...
23 struct __wait_queue {
24     unsigned int flags;
25     #define WQ_FLAG_EXCLUSIVE 0x01
26     struct task_struct * task;
27     wait_queue_func_t func;
28     struct list_head task_list;
29 };
30
31 struct __wait_queue_head {
32     spinlock_t lock;
33     struct list_head task_list;
34 };
35 typedef struct __wait_queue_head wait_queue_head_t;

```

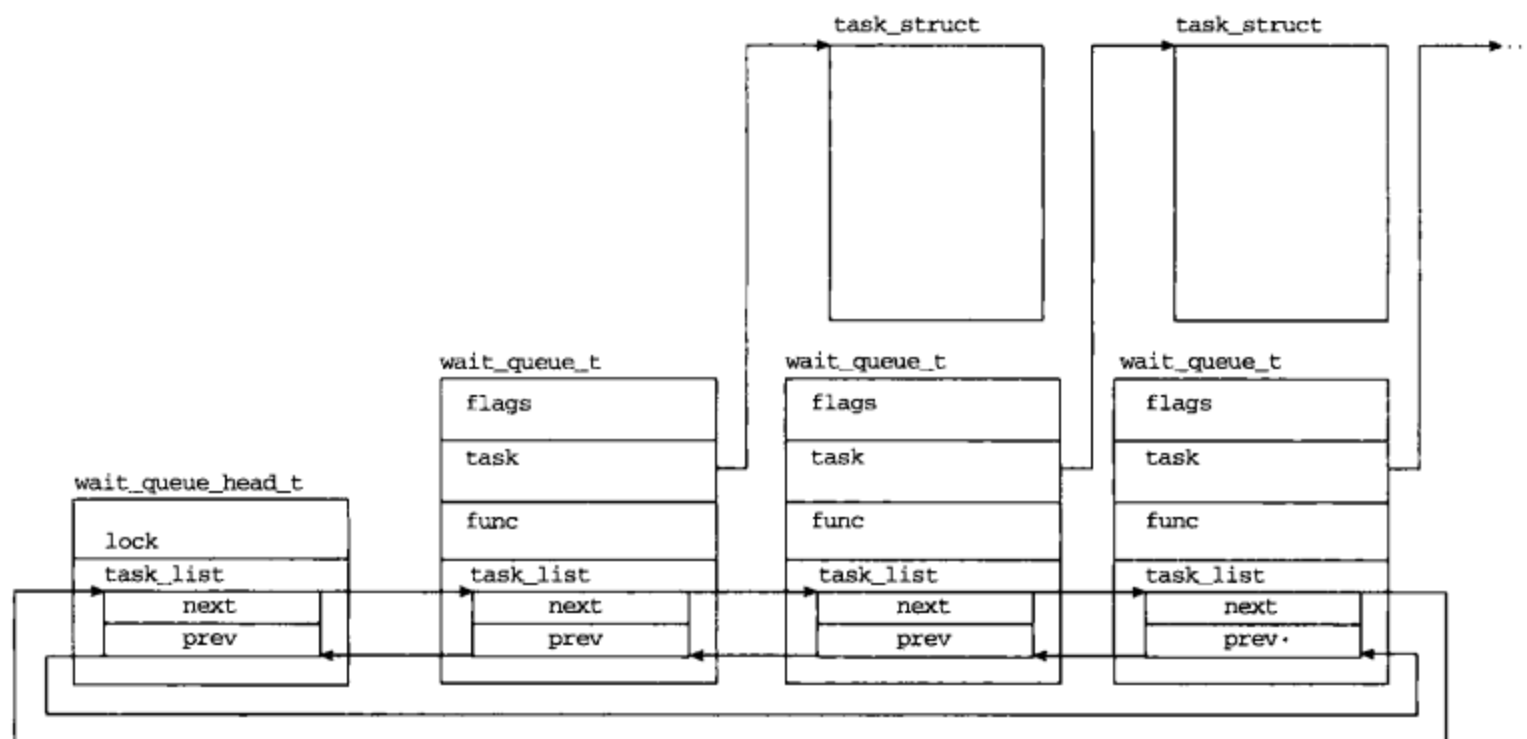


图 3-14 等待队列的结构

wait_queue_t 结构由下列字段组成。

- ❑ **flags**。可以存放值 WQ_FLAG_EXCLUSIVE (被设置为 1) 或 ~WQ_FLAG_EXCLUSIVE (被设置为 0)。WQ_FLAG_EXCLUSIVE 标志标记这个进程为独占式进程。独占式进程和非独占式进程将在下一节中讨论。
- ❑ **task**。一个指针，指向被加入到等待队列上的进程的进程描述符。
- ❑ **func**。存放函数的一个结构，该函数用于唤醒等待队列上的进程。这个字段的默认值为 default_wake_function()，该函数将在 3.7.2 节中详细介绍。

wait_queue_func_t 定义如下：

```

-----
include/linux/wait.h
typedef int (*wait_queue_func_t)(wait_queue_t *wait,
unsigned mode, int sync);
-----

```


此处, `wait` 是指向等待队列的指针, 而 `mode` 是 `TASK_INTERRUPTIBLE` 或者 `TASK_UNINTERRUPTIBLE`, `sync` 表示唤醒是否应被同步。

□ **task_list**。这个结构包含了两个指针, 分别指向等待队列中的前一个进程和后一个进程。

`__wait_queue_head` 结构是等待队列链表的表头, 由下列字段组成:

□ **lock**。每个链表有一个锁, 这使得向等待队列添加或删除数据项时能够同步。

□ **task_list**。这个结构指向等待队列中第一个进程和最后一个进程。

10.1.4 节描述了等待队列实现的例子。通常, 进程让自己睡眠涉及对某个 `wait_event*` 宏的调用 (稍后讨论), 或者, 如同第 10 章所举的例子一样, 可以通过执行下列步骤来实现。

(1) 通过声明等待队列, 进程利用 `DECLARE_WAITQUEUE_HEAD` 继续睡眠。

(2) 利用 `add_wait_queue()` 或 `add_wait_queue_exclusive()` 把自己加入等待队列。

(3) 把进程状态变为 `TASK_INTERRUPTIBLE` 或 `TASK_UNINTERRUPTIBLE`。

(4) 检测外部事件, 如果外部事件还没有发生, 则调用 `schedule()`。

(5) 外部事件发生后, 把进程设置为 `TASK_RUNNING` 状态。

(6) 通过调用 `remove_wait_queue()` 从等待队列删除自己。

通过调用某个 `wake_up` 宏来唤醒进程。这些宏唤醒某个等待队列上的所有进程。它把进程置为 `TASK_RUNNING` 状态并放回运行队列。

让我们来看看调用函数 `add_wait_queue()` 时发生了什么。

3.7.1 添加到等待队列

有两个不同的函数可以向等待队列添加睡眠进程, 这就是函数 `add_wait_queue()` 和 `add_wait_queue_exclusive()`, 它们分别对应两种类型的睡眠进程。非独占式等待进程是所等待的条件返回时不被其他进程所共享, 而独占式等待进程则在等待一个其他进程可能正在等待的条件, 这就可能会产生一个竞态条件。

`add_wait_queue()` 函数向等待队列插入一个非独占式进程。非独占式进程是指在任何条件下, 当等待的事件完成时就被内核唤醒的进程。这个函数设置等待队列结构的 `flags` 字段, 表示把睡眠进程的标志的相应位设为 0, 同时设置等待队列锁, 以避免中断访问同一个等待队列而产生竞争条件, 之后把 `wait_queue_t` 结构加入等待队列链表, 并且从等待队列恢复锁, 使得其他进程可以使用。

```
-----
kernel/fork.c
93 void add_wait_queue(wait_queue_head_t *q, wait_queue_t * wait)
94 {
95     unsigned long flags;
96
97     wait->flags &= ~WQ_FLAG_EXCLUSIVE;
98     spin_lock_irqsave(&q->lock, flags);
99     __add_wait_queue(q, wait);
100     spin_unlock_irqrestore(&q->lock, flags);
101 }
```

`add_wait_queue_exclusive()` 函数向等待队列插入一个独占式进程。它把等待队列结构的 `flags` 字段的相应位设置为 1, 并且以与 `add_wait_queue()` 大致相同的独占方式进行操作, 但有一个例外, 它将独占进程添加到队列的末尾。这意味着在一个特定的等待队列里, 非独占式进程在前面, 独占式进程在后面。与讨论唤醒进程时所看到的一样, 这个次序也就是等待队列中的进程被唤醒的次序。

```
-----
kernel/fork.c
105 void add_wait_queue_exclusive(wait_queue_head_t *q, wait_queue_t * wait)
106 {
107     unsigned long flags;
108
109     wait->flags |= WQ_FLAG_EXCLUSIVE;
110     spin_lock_irqsave(&q->lock, flags);
111     __add_wait_queue_tail(q, wait);
112     spin_unlock_irqrestore(&q->lock, flags);
113 }
-----
```

3.7.2 等待事件

虽然在 Linux 2.6 内核中仍然支持 `sleep_on()`、`sleep_on_timeout()` 和 `interruptible_sleep_on()` 接口, 但是在 2.7 版本中将会去掉这几个接口。因此, 此处只介绍将要取代 `sleep_on*`() 接口的 `wait_event*`() 接口。

`wait_event*`() 接口包括 `wait_event()`、`wait_event_interruptible()` 和 `wait_event_interruptible_timeout()`。图 3-15 显示了这些函数的基本调用轨迹。

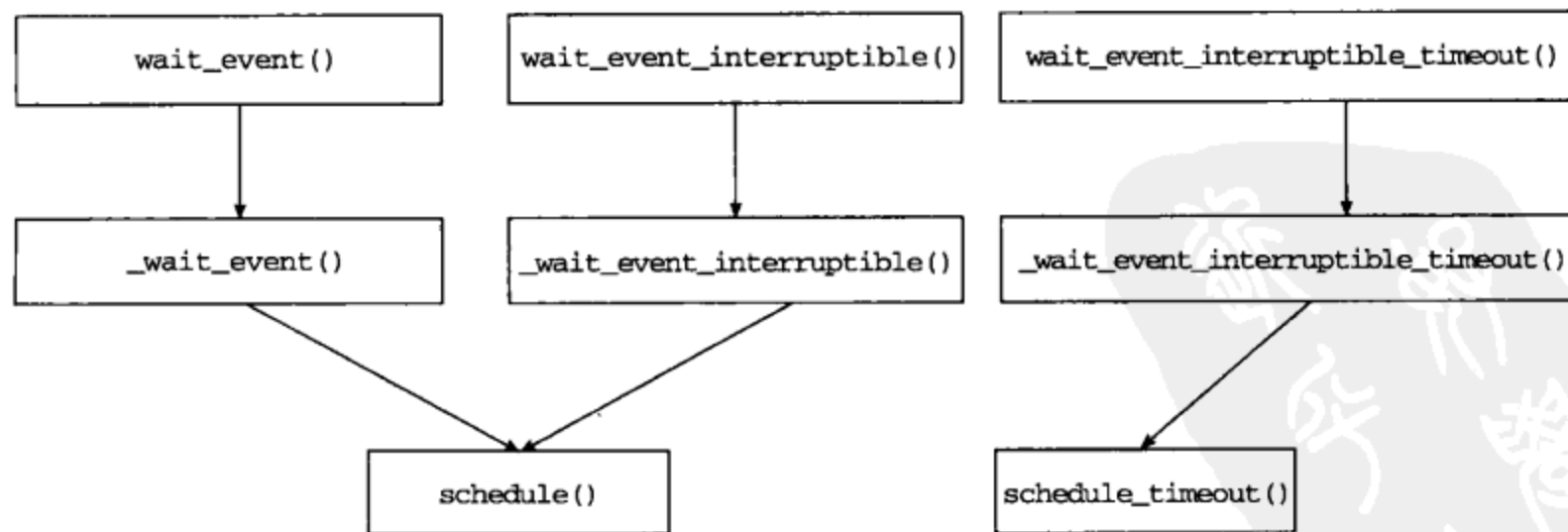


图 3-15 `wait_event*`() 的调用图

本节将详细分析并描述与 `wait_event()` 相关的接口, 同时也会提及相对于其他两个函数有什么差别。`wait_event()` 接口是对 `__wait_event()` 调用以无限循环的方式进行包装, 仅当所等待的条件返回时, 才退出循环。`wait_event_interruptible_timeout()` 传递的第三个参数是 `int` 类型的 `ret`, 用于传递超时时间。

`wait_event_interruptible()`是这三个接口中唯一有返回值的接口。如果一个信号中断了等待事件，这个返回值就是`-ERESTARTSYS`，否则，如果条件成立，就返回0。

```
-----
include/linux/wait.h
137 #define wait_event(wq, condition)
138 do {
139     if (condition)
140         break;
141     __wait_event(wq, condition);
142 } while (0)
-----
```

`__wait_event()`接口完成所有围绕进程状态改变和描述符操作的工作。

```
-----
include/linux/wait.h
121 #define __wait_event(wq, condition)
122 do {
123     wait_queue_t __wait;
124     init_waitqueue_entry(&__wait, current);
125
126     add_wait_queue(&wq, &__wait);
127     for (;;) {
128         set_current_state(TASK_UNINTERRUPTIBLE);
129         if (condition)
130             break;
131         schedule();
132     }
133     current->state = TASK_RUNNING;
134     remove_wait_queue(&wq, &__wait);
135 } while (0)
-----
```

第124~126行：为当前进程初始化等待队列描述符，并把该描述符添加到上层调用传递过来的等待队列中。到目前为止，`__wait_event_interruptible`和`__wait_event_interruptible_timeout`看起来与`__wait_event`完全相同。

第127~132行：这几行代码设置一个无限循环，仅当条件成立时才退出循环。进程在这个条件上阻塞前，利用`set_current_state`宏将其状态设置为`TASK_INTERRUPTIBLE`。回想一下，这个宏引用了指向当前进程的指针，因此不必把进程信息传递给它。该进程阻塞以后，它就通过调用`schedule()`将CPU让给其他进程。在这一点上，`__wait_event_interruptible()`与它有很大的不同；该函数把进程的状态字段设置成`TASK_UNINTERRUPTIBLE`，并让它等待`signal_pending`调用的返回，`signal_pending`调用用于等待发送给当前进程的信号。`__wait_event_interruptible_timeout`与`__wait_event_interruptible`非常类似，区别是：当调用调度程序时，前者调用`schedule_timeout()`，而不是`schedule()`。`schedule_timeout`把传递给原始接口`wait_event_interruptible_timeout`的超时长度参数作为自己的参数。

第133~134行：此处，条件已经满足，或者如果是在另外两个接口的情况下，就意味着进程可能收到一个信号或者已经超时。现在，将进程描述符的`state`字段设置回`TASK_RUNNING`（调

度程序把它放入运行队列)。最后，从等待队列中删除描述符`_wait`。`remove_wait_queue()`函数会在删除描述符前为等待队列上锁，而在返回以前再为它解锁。

3.7.3 唤醒进程

进程必须被唤醒以检查它等待的条件是否成立。注意，进程可以让自己睡眠，但是不能唤醒自己。除了有三个主要的“唤醒”函数以外，还有许多宏可以用来“唤醒”等待队列中的进程。宏`wake_up`、`wake_up_nr`、`wake_up_all`、`wake_up_interruptible`、`wake_up_interruptible_nr`和`wake_up_interruptible_all`都以不同的参数来调用`__wake_up()`。宏`wake_up_all_sync`和`wake_up_interruptible_sync`都以不同的参数来调用`__wake_up_sync()`。最后，`wake_up_locked`宏默认为`__wake_up_locked()`函数：

```
-----
include/linux/wait.h
116 extern void FASTCALL(__wake_up(wait_queue_head_t *q, unsigned int mode, int nr));
117 extern void FASTCALL(__wake_up_locked(wait_queue_head_t *q, unsigned int mode));
118 extern void FASTCALL(__wake_up_sync(wait_queue_head_t *q, unsigned int mode, int
    nr));
119
120 #define wake_up(x)    __wake_up((x), TASK_UNINTERRUPTIBLE | TASK_INTERRUPTIBLE, 1)
121 #define wake_up_nr(x, nr) __wake_up((x), TASK_UNINTERRUPTIBLE | TASK_INTERRUPTIBLE, nr)
122 #define wake_up_all(x) __wake_up((x), TASK_UNINTERRUPTIBLE | TASK_INTERRUPTIBLE, 0)
123 #define wake_up_all_sync(x) __wake_up_sync((x), TASK_UNINTERRUPTIBLE |
    TASK_INTERRUPTIBLE, 0)
124 #define wake_up_interruptible(x) __wake_up((x), TASK_INTERRUPTIBLE, 1)
125 #define wake_up_interruptible_nr(x, nr) __wake_up((x), TASK_INTERRUPTIBLE, nr)
126 #define wake_up_interruptible_all(x) __wake_up((x), TASK_INTERRUPTIBLE, 0)
127 #define wake_up_locked(x)    __wake_up_locked((x), TASK_UNINTERRUPTIBLE |
    TASK_INTERRUPTIBLE)
128 #define wake_up_interruptible_sync(x) __wake_up_sync((x), TASK_INTERRUPTIBLE, 1
129 )
-----
```

让我们来看看`__wake_up()`：

```
-----
kernel/sched.c
2336 void fastcall __wake_up(wait_queue_head_t *q, unsigned int mode, int
    nr_exclusive)
2337 {
2338     unsigned long flags;
2339
2340     spin_lock_irqsave(&q->lock, flags);
2341     __wake_up_common(q, mode, nr_exclusive, 0);
2342     spin_unlock_irqrestore(&q->lock, flags);
2343 }
-----
```

第 2336 行：传给`__wake_up`的参数包括：**q**，一个指向等待队列的指针；**mode**，表示将要唤醒的线程类型（由线程的状态来标识）；**nr_exclusive**，表示是独占式唤醒还是非独占式唤醒。

独占式唤醒 (`nr_exclusive = 0` 时) 将唤醒等待队列中的所有进程 (包括独占式进程和非独占式进程), 而非独占式唤醒只唤醒一个独占式进程和所有的非独占式进程。

第 2340 和 2342 行: 这两行分别设置和解除等待队列的自旋锁。在调用 `__wake_up_common()` 之前设置该锁, 以确保不会出现竞态条件。

第 2341 行: 函数 `__wake_up_common` 完成 `wakeup` 函数的大部分工作:

```
-----
kernel/sched.c
2313 static void __wake_up_common(wait_queue_head_t *q,
unsigned int mode, int nr_exclusive, int sync)
2314 {
2315     struct list_head *tmp, *next;
2316
2317     list_for_each_safe(tmp, next, &q->task_list) {
2318         wait_queue_t *curr;
2319         unsigned flags;
2320         curr = list_entry(tmp, wait_queue_t, task_list);
2321         flags = curr->flags;
2322         if (curr->func(curr, mode, sync) &&
2323             (flags & WQ_FLAG_EXCLUSIVE) &&
2324             !--nr_exclusive)
2325             break;
2326     }
2327 }
```

第 2313 行: 传给 `__wake_up_common` 的参数分别是: **q**, 一个指向等待队列的指针; **mode**, 将要唤醒的线程类型; **nr_exclusive**, 如前所示的唤醒类型; **sync**, 表示唤醒是否应该同步。

第 2315 行: 此处为链表元素操作设置临时指针。

第 2317 行: `list_for_each_safe` 宏扫描等待队列的每一项, 这是循环的开始。

第 2320 行: `list_entry` 宏返回 `tmp` 变量所存放的等待队列结构的地址。

第 2322 行: 调用 `wait_queue_t` 的 `func` 字段。默认情况下, 它调用如下所示的 `default_wake_function()`:

```
-----
kernel/sched.c
2296 int default_wake_function(wait_queue_t *curr, unsigned mode,
int sync)
2297 {
2298     task_t *p = curr->task;
2299     return try_to_wake_up(p, mode, sync);
2300 }
```

这个函数对 `wait_queue_t` 结构所指向的进程调用 `try_to_wake_up()` (参见 `kernel/sched.c`)。它完成唤醒进程的大部分工作, 包括把进程放入运行队列。

第 2322~2325 行: 如果被唤醒的进程是第一个独占式进程, 则终止循环。如果知道所有独占式进程都排在等待队列末尾的话, 这将是很有意义的。当在等待队列中遇到第一个独占式进程后,

剩余的所有进程也将是独占式的，因此，不应该唤醒它们，而应跳出循环。

3.8 异步执行流程

前面已经介绍过，进程能够通过中断从一个状态转换到另一个状态，例如从 `TASK_INTERRUPTIBLE` 到 `TASK_RUNNING`。获得这种转换的途径之一是包括异常和中断在内的异步执行。同时，进程也可以在用户态和内核态之间来回切换。接下来首先介绍异常是如何工作的，之后，再来分析中断究竟是如何工作的。

3.8.1 异常

异常 (exception)，也称作**同步中断** (synchronous interrupts)，是完全发生在处理器硬件内部的事件。这些事件和处理器的执行是同步的，也就是说，它们不是发生在代码指令执行期间，而是发生在代码指令执行之后。处理器异常的例子包括对一个没有映射到物理内存的虚拟存储单元的引用 (称为**缺页**)，以及导致被 0 除的计算。要强调的一点是，异常 (有时也称为软中断 `irq`) 通常发生在指令执行之后，这使得它们与外部事件或**异步事件**有很大的区别，稍后将会在 3.8.2 节中详细讨论。

大多数现代处理器 (包括 x86 和 PPC) 允许程序员执行某些指令来产生一个异常。这些指令可以被看做是用来辅助硬件的子例程调用，例如**系统调用**。

系统调用

Linux 为用户态的程序提供了进入内核的入口点，可以通过这个入口点从内核请求服务或访问硬件。这些入口点在内核中是经过标准化的、预先定义好的。用户态的程序可用的许多 C 库例程，例如图 3-9 中的 `fork()` 函数，把代码和一个或多个系统调用捆绑在一起形成一个单独的函数。当用户进程调用其中某个函数时，相应的值被放入适当的处理器寄存器中，并产生一个软中断。然后，再由这个软中断来调用内核入口点。系统调用 (syscalls) 也可以由内核代码来访问，尽管不推荐这样做。从何处访问系统调用是我们讨论的源点，因为从内核调用系统调用可以提高系统性能。这种性能上的提高要和增加的复杂性以及代码的可维护性进行权衡。本节讨论“传统的”从用户空间调用系统调用的实现过程。

系统调用能够在用户空间和内核空间之间传送数据。内核提供了两个函数来完成这种数据传送：`copy_to_user()` 和 `copy_from_user()`。和所有内核编程一样，传送数据时，指针、长度、描述符和许可权的有效性是很关键的。这些函数都内置了有效性的检验。有趣的是，它们返回的是未被传送的字节数。

系统调用自身的特点决定了它的实现是与硬件相关的。在传统的 Intel 体系结构中，所有的系统调用都使用软中断 `0x80`^①。

将系统调用的参数传入通用寄存器，其唯一的系统调用号被放入 `%eax`。系统调用在 x86 体系

① 新的 (PIV+) Intel 处理器在提高性能方面做了许多努力，其中大部分工作随着 `vsyscalls` 的实现而得以完成。`vsyscalls` 基于对用户空间内存 (特别是“`vsyscall`”页) 的访问，它使用了比传统的 `int 0x80` 调用更快的 `sysenter` 和 `sysexit` 指令 (当可用时)。在许多 PPC 的实现中也追求类似的性能提高。

结构中的实现把参数个数限制为 5 个。如果函数的参数多于 5 个，可以给函数传递一个指向参数块的指针。在执行汇编指令 `int 0x80` 时，一个特殊的内核态例程将借助处理器的异常处理能力被调用执行。下面来看一个如何初始化系统调用入口的例子：

```
set_system_gate(SYSCALL_VECTOR, &system_call);
```

这个宏在入口 128 (`SYSCALL_VECTOR`) 处创建一个用户特权描述符，指向 `entry.S` 中系统调用处理程序 (`system_call`) 的地址。

和我们在下一节中将看到的一样，PPC 中断例程被“固定”到某些存储单元；外部中断处理程序被固定到地址 `0x500` 处，系统定时器被固定到地址 `0x900` 处，等等。系统调用指令 `sc` 指向地址 `0xc00`。接下来分析一下 `head.S` 中的代码片断，这里的处理程序是为 PPC 系统调用设置的。

```
-----
arch/ppc/kernel/head.S
484  /* System call */
485  . = 0xc00
486  SystemCall:
487  EXCEPTION_PROLOG
488  EXC_XFER_EE_LITE(0xc00, DoSyscall)
-----
```

第 485 行：地址的定位。这一行告诉装载程序下一条指令将被定位在地址 `0xc00` 处。由于标号遵循类似的规则，因此，标号 `SystemCall` 连同宏 `EXCEPTION_PROLOG` 的第一行代码都从地址 `0xc00` 处开始。

第 488 行：这个宏分派 `DoSyscall()` 处理程序。

在 x86 和 PPC 这两种体系结构中，系统调用号和所有的参数都是存入处理器的寄存器中。

当 x86 的异常处理程序处理 `int0x80` 时，将对系统调用表进行索引。文件 `arch/i386/kernel/entry.S` 中包含了低级中断处理例程和系统调用表 `sys_call_table`。同样，对于 PPC 而言，系统调用的低级中断处理例程是在 `arch/ppc/kernel/entry.S` 中定义的，而 `sys_call_table` 则在 `arch/ppc/kernel/misc.S` 中。

系统调用表是用汇编代码实现的 C 数组，每个表项占 4 字节，每项被初始化为一个函数的地址。按照惯例，必须用“`sys_`”预先定义好函数名，因为表中的位置决定了系统调用号，因此必须把函数名称添加到表的尾部。即使使用不同的汇编语言，不同体系结构的系统调用表也几乎是完全一样的。然而，直到本书撰写之时，PPC 的系统调用表还只有 255 个表项，而 x86 的系统调用表有 275 个表项。

文件 `include/asm-i386/unistd.h` 和 `include/asm-ppc/unistd.h` 把系统调用和它在系统调用表中的位置号相关联。在该文件中，“`sys_`”被替换成“`__NR_`”，同时还包含辅助用户程序把参数装入寄存器的宏（2.3 节介绍了 C 和汇编变量以及内嵌汇编程序）。

接下来看一下如何添加一个名为 `sys_ourcall` 的系统调用。系统调用必须被添加到 `sys_call_table`。下列代码就是把系统调用加入 x86 的 `sys_call_table`：


```

-----
arch/i386/kernel/entry.S
607  .data
608  ENTRY(sys_call_table)
609  .long sys_restart_syscall /* 0 - old "setup()" system call, used for restarting*/
...
878  .long sys_tgkill /* 270 */
879  .long sys_utimes
880  .long sys_fadvise64_64
881  .long sys_ni_syscall /* sys_vserver */
882  .long sys_ourcall /* our syscall will be 274 */
883
884  nr_syscalls=(.-sys_call_table)/4
-----

```

在 x86 体系结构中，该系统调用编号为 274。若要在 PPC 中添加名为 sys_ourcall 的系统调用，其入口将是第 255 号。此处，当把系统调用及其位置号的关联引入文件 include/asm-ppc/unistd.h 时，会看到它是什么样子的。__NR_ourcall 就是入口号 255，位于系统调用表的表尾。

```

-----
include/asm-ppc/unistd.h
/*
 * This file contains the system call numbers.
 */

#define __NR_restart_syscall 0
#define __NR_exit 1
#define __NR_fork 2
...
#define __NR_utimes 271
#define __NR_fadvise64_64 272
#define __NR_vserver 273
#define __NR_ourcall 274

/* #define NR_syscalls 274 this is the old value before our syscall */
#define NR_syscalls 275
-----

```

下一节将讨论中断和通知内核需要处理中断时所涉及的硬件。作为与中断稍有差异的异常，其处理程序将会在适当的时候被调用。尽管异常和中断在处理时采用的方法相同，但异常趋向于给当前进程发回信号，而不是作用于硬件设备。

3.8.2 中断

中断和处理器的执行是异步的，这意味着中断发生在两条指令之间。处理器通过传到其引脚（INTR 或者 NMI）的外部信号来接收中断通知。这个信号来自于称为中断控制器的硬件设备。中断和中断处理器都是与硬件及系统相关的。不同体系结构对中断控制器的设计有很大不同。本节简要介绍独立于体系结构的部分和依赖于体系结构的部分在硬件方面的主要差异，并简单分析用于追踪内核代码的函数。

由于在任何给定的时刻，处理器必须与若干外部设备中的某一个进行通信，因而中断控制器是必不可少的。早期的 x86 计算机使用了一对级联的 Intel 8259 中断处理器^①，这些中断处理器通过配置，能够辨别 15 根离散的中断线（IRQ）（参见图 3-16）。当中断控制器有一个待处理的中断（例如，当你按下一个键）时，它将触发连接到处理器上的 INTA 线。然后，处理器通过激活连接到中断控制器 INTA 线上的应答线来应答这个信号。此时，中断控制器才把 IRQ 数据传送到处理器。以上过程被称为一个中断应答周期。

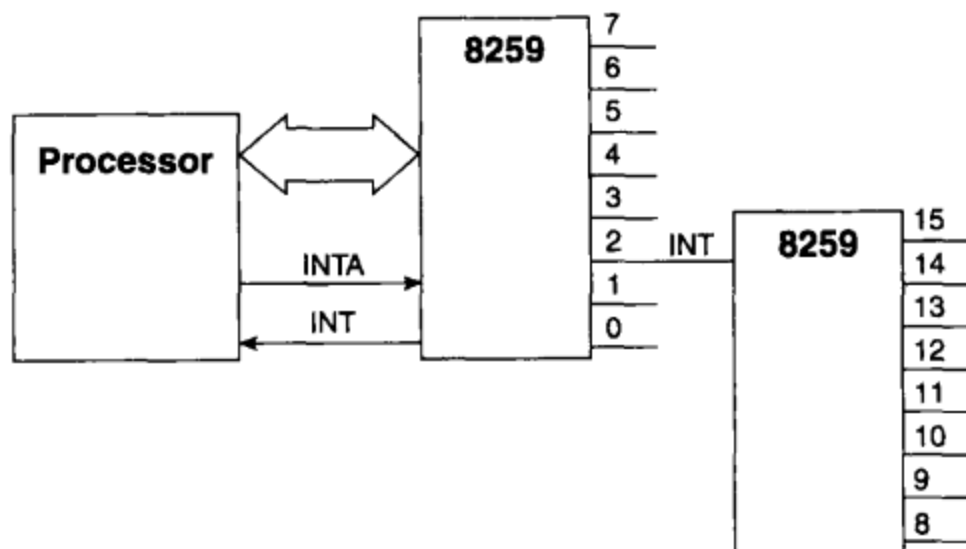


图 3-16 级联的中断控制器

更新的 x86 处理器拥有一个本地的 APIC（Advanced Programmable Interrupt Controller，高级可编程中断控制器）。本地 APIC（被嵌入在处理器包中）接收来自以下中断源的中断信号：

- ☐ 处理器的中断引脚（LINT0, LINT1）；
- ☐ 内部定时器；
- ☐ 内部性能监控器；
- ☐ 内部温度传感器；
- ☐ 内部 APIC 错误；
- ☐ 另外一个处理器（用于处理器间的中断）；
- ☐ 外部 I/O APIC（经由多处理器系统上的 APIC 总线）。

APIC 收到一个中断信号后，就把这个信号交给处理器核心（在处理器包中）。图 3-17 中显示的 I/O APIC 是处理器芯片集的一部分，它是为接收 24 个可编程的中断输入而设计的。

对于具有本地 APIC 的 x86 处理器，你也可以将其（中断模式）配置成传统的 8259 类型的中断控制器，而不使用 I/O APIC 体系架构（或者说 I/O APIC 可以被配置成 8259 控制器的接口，以 8259 的方式使用）。为了查明一个系统是否在使用 I/O APIC 体系结构，可在命令行输入如下命令：

```
lkp:~# cat /proc/interrupts
```

如果看到了列出的 I/O-APIC，说明 I/O APIC 正在使用，如果看到的是 XT-PIC，就意味着正在使用 8259 类型的体系结构。

^① 第二个 8259 的输出连接到第一个 8259 的 IRQ（通常为 IRQ2）上。

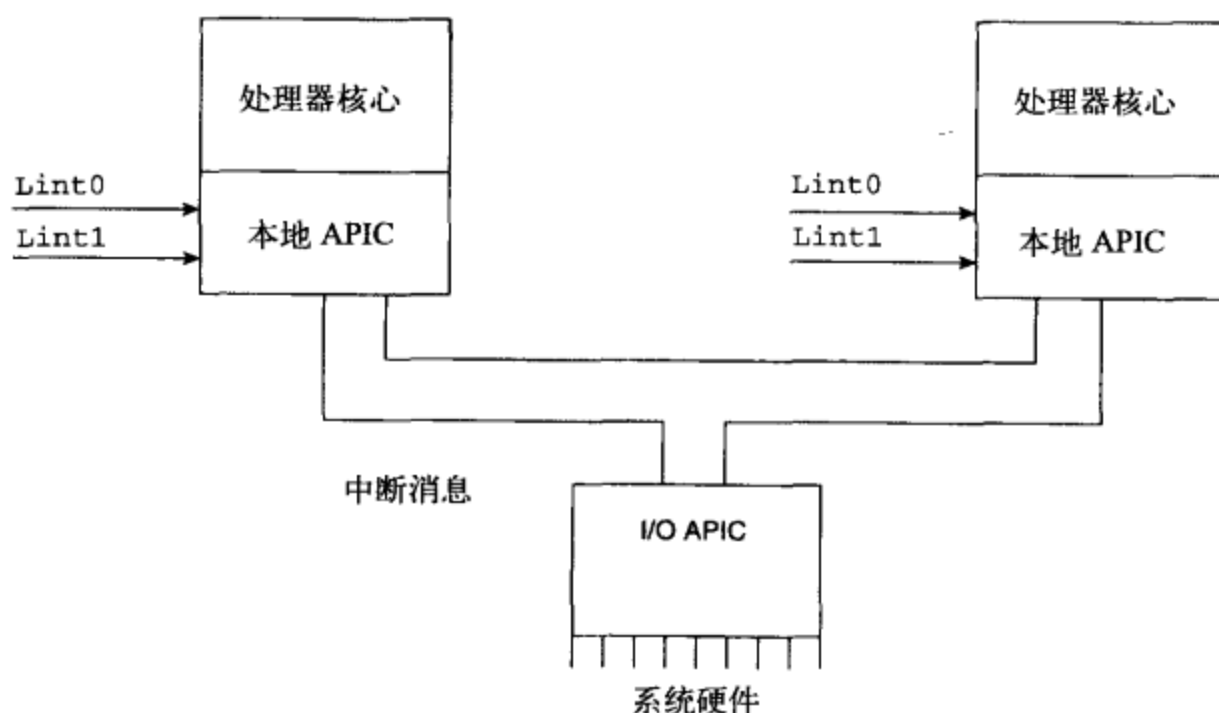


图 3-17 I/O APIC

Power Mac G4 和 G5 的 PowerPC 中断控制器被集成到 Key Largo 和 K2 I/O 控制器中。可在命令行输入这样的命令：

```
lkp:~# cat /proc/interrupts
```

在 G4 机上产生了 OpenPIC，它是一个由 AMD 和 Cyrix 于 1995 年为多处理器系统发起的开放可编程中断控制器（Open Programmable Interrupt Controller, OpenPIC）标准。MPIC 是 IBM 对 OpenPIC 的实现，并用在他们的几个公共硬件平台参数（CHRP）设计中。以前的苹果机有一个用于 4xx 嵌入式处理器的内部中断控制器，中断控制器的核心被集成到 ASIC 芯片中。

既然已经讨论过硬件为什么、如何以及何时给内核传送中断请求，接下来就可以分析内核处理硬件系统时钟中断的这个实际例子，并详细叙述中断将在何处被传送。在分析系统定时器的代码时，可以看到，在 x86 和 PPC 体系结构中中断发生时，硬件到软件的接口都是用跳转表来实现的，跳转表用来为给定的中断选择相应的处理程序。

x86 体系结构的每个中断都被赋予一个唯一的编号或者向量。中断发生时，就用这个向量来索引 IDT（Interrupt Descriptor Table，中断描述符表），可参阅 x86 门描述符格式的《Intel 程序员参考资料》。IDT 允许硬件在中断时辅助软件进行地址解析，并检查处理程序代码的特权。PPC 体系结构中的处理稍有不同，它使用编译时创建的中断向量表来执行适当的中断处理程序。（稍后在比较 x86 和 PPC 的系统定时器的中断处理过程时，将会看到这两种体系结构在软件的初始化和跳转表的使用上有更多的不同。）下一节讨论中断处理程序及其实现。接下来，作为 Linux 中断及其相关处理程序实现的一个例子，将讨论系统定时器。

首先来看看不同种类的中断处理程序。

1. 中断处理程序

中断和异常处理程序很像常规的 C 函数。它们可能并经常调用硬件特有的汇编例程。Linux 中断处理程序划分为高性能的上半部分和低性能的下半部分。

□ 上半部分 (top half)。这一部分必须尽快执行。上半部分处理程序可以在所有本地中断（对

给定处理器来说)禁用的情况下运行(也叫快速处理程序),这依赖于它们如何被注册。上半部分中的代码必须仅限于直接响应硬件或者执行紧迫的进程。在上半部分中拖延时间会大大影响系统的性能。为了保持高性能和低延迟(响应设备所花的时间),引入了下半部分体系结构。

- 下半部分(bottom half)。这一部分允许处理程序编写者把不太紧急的工作推后,直到内核有充裕的时间来处理^①。请注意,中断随着系统的执行异步地到达;此刻,内核也许正在做更紧迫的事情。有了下半部分体系结构,中断处理程序编写者可以让内核稍迟一点运行不太紧急的处理程序代码。

表3-8解释了下半部分中断处理的4个最普通的方法。

表3-8 下半部分中断处理的方法

“陈旧的”下半部分	无论处理器的个数有多少,每次只有一个下半部分能够运行,因此,这些每个SMP上都有下半部分处理程序被逐步淘汰。这种系统已经在2.6内核中删除,只在参考时提及
工作队列	据说上半部分代码运行在中断上下文中,这意味着它不 and 任何进程关联。由于没有进程关联,代码就不能睡眠或阻塞。工作队列运行在进程上下文中,并具有任何内核线程具备的能力。它有一个丰富的函数集,用于创建、调度、撤销进程,等等。有关工作队列的更多信息,参见10.1.5节
软中断	软中断也运行在中断上下文中,它类似于下半部分,但有一个例外,那就是同一类型的软中断能够同时运行在多个处理器上。系统中只有32个软中断。系统定时器就使用软中断
tasklet	类似于软中断,但不存在限制。所有的tasklet都通过一个软中断来产生,同一个tasklet不能在多个处理器上同时运行。和软中断相比,tasklet的接口更容易实现和使用

2. IRQ结构

有三个主要的结构包含了与IRQ相关的所有信息:irq_desc_t、irqaction和hw_interrupt_type。图3-18解释了它们之间是如何相互关联的。

● irq_desc_t结构

irq_desc_t结构是主要的IRQ描述符,它存放在大小为NR_IRQS(它的值依赖于体系结构)的全局数组irq_desc中。

```

-----
include/linux/irq.h
60 typedef struct irq_desc {
61     unsigned int status; /* IRQ 状态 */
62     hw_irq_controller *handler;
63     struct irqaction *action; /* IRQ 操作链表 */
64     unsigned int depth; /* 禁止嵌套的 irq */
65     unsigned int irq_count; /* 用于检查所打断的中断 */
66     unsigned int irqs_unhandled;
67     spinlock_t lock;
68 } ____cacheline_aligned irq_desc_t;
69
70 extern irq_desc_t irq_desc [NR_IRQS];
-----

```

① Linux的早期版本为系统定时器使用了上半部分/下半部分处理程序。后来,对此进行了重写,只保留了高性能的上半部分。

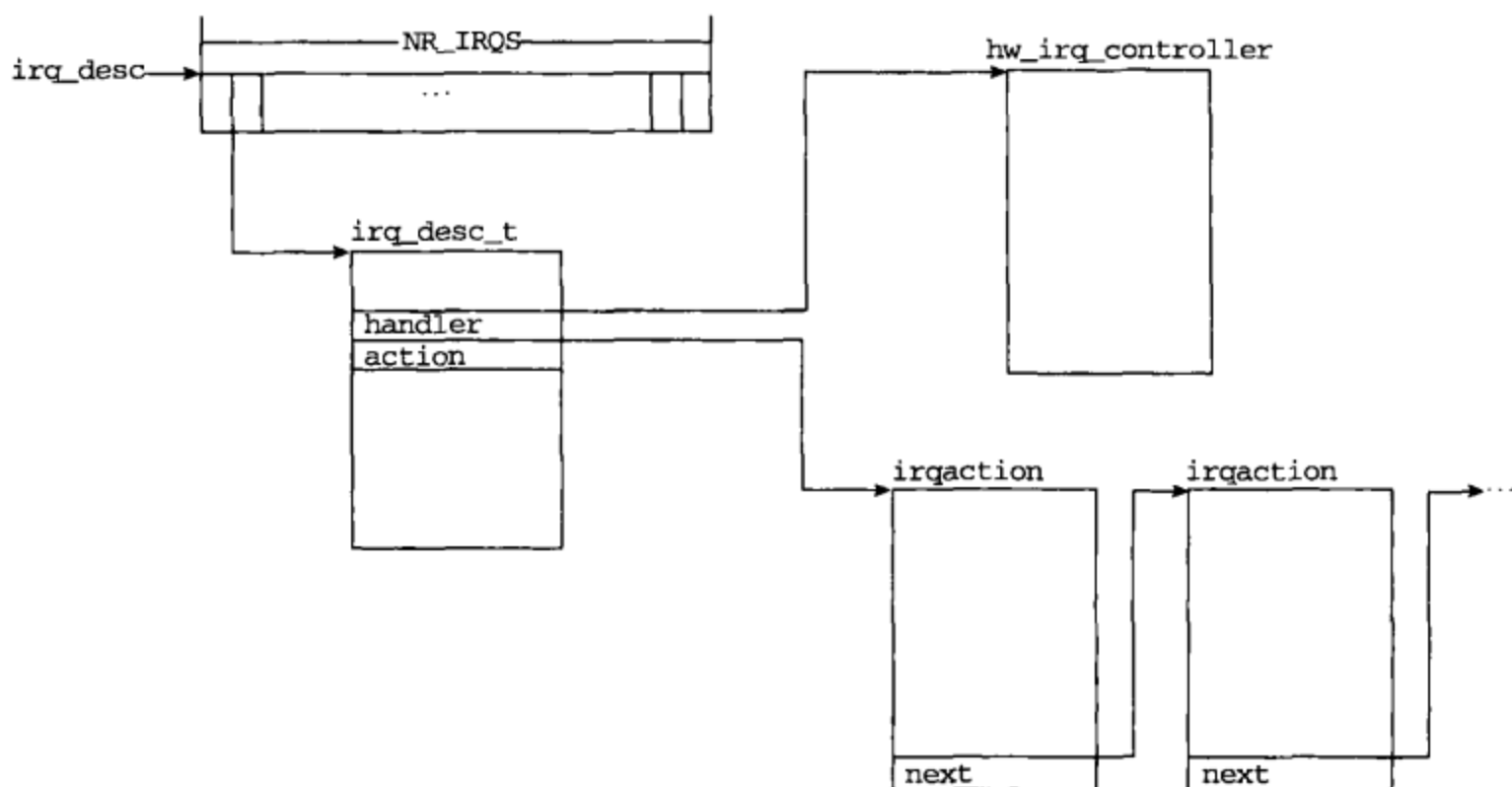


图 3-18 IRQ 结构

第 61 行：设置标志来描述 IRQ 线的状态，从而确定 status 字段的值。表 3-9 给出了这些标志。

表3-9 irq_desc_t->field标志

标 志	说 明
IRQ_INPROGRESS	表示正在为IRQ线执行处理程序
IRQ_DISABLED	表示IRQ被软件禁止，因此，即使启用了物理请求线本身，它的处理程序也不会被执行
IRQ_PENDING	中间状态，表示中断的发生已被确认，但是处理程序还没有执行
IRQ_REPLAY	前一个IRQ还没有被确认
IRQ_AUTODETECT	当IRQ线被探测时所设置的状态
IRQ_WAITING	探测时使用
IRQ_LEVEL	IRQ是电平触发，而不是边沿触发的
IRQ_MASKED	这个标志在内核代码中不使用
IRQ_PER_CPU	用于表明IRQ线对CPU调用是局部的

第 62 行：handler 字段是指向 hw_irq_controller 的一个指针。hw_irq_controller 是对 hw_interrupt_type 结构的类型定义，是用来描述低层硬件的中断控制器描述符。

第 63 行：action 字段包含一个指向 irqaction 结构的指针。当 IRQ 被启用时，记录中断处理程序例程的这个结构将被执行，稍后将详细介绍这个结构。

第 64 行：depth 字段是禁止 IRQ 的嵌套计数器。只有当这个字段的值为 0 时，IRQ_DISABLE 才被清零。

第 65~66 行：irq_count 字段和 irqs_unhandled 字段共同确定可能被延迟的中断请求。它们在 x86 和 PPC64 的函数 note_interrupt() (arch/<arch>/kernel/irq.c) 中使用。

第 67 行：lock 字段存放描述符的自旋锁。

● irqaction 结构

内核使用 irqaction 结构来记录中断处理程序及其与 IRQ 的关联。接下来看看下一节将要讨论的它的结构和字段：

```
-----
include/linux/interrupt.h
35 struct irqaction {
36     irqreturn_t (*handler) (int, void *, struct pt_regs *);
37     unsigned long flags;
38     unsigned long mask;
39     const char *name;
40     void *dev_id;
41     struct irqaction *next;
42 };
-----
```

第 36 行：**handler** 字段是指向中断处理程序的指针，当发生相应的中断时就调用这个处理程序。

第 37 行：flags 字段包含了 SA_INTERRUPT 之类的标志，表示中断处理程序将在所有中断禁用的情况下运行，SA_SHIRQ 表示处理程序可以和其他处理程序共享一根 IRQ 线。

第 39 行：name 字段用于存放被注册的中断名称。

● hw_interrupt_type 结构

hw_interrupt_type 或 hw_irq_controller 包含了与系统的中断控制器相关的所有数据。首先来先分析这个结构，然后再讨论如何为一对中断控制器实现该结构。

```
-----
include/linux/irq.h
40 struct hw_interrupt_type {
41     const char * typename;
42     unsigned int (*startup)(unsigned int irq);
43     void (*shutdown)(unsigned int irq);
44     void (*enable)(unsigned int irq);
45     void (*disable)(unsigned int irq);
46     void (*ack)(unsigned int irq);
47     void (*end)(unsigned int irq);
48     void (*set_affinity)(unsigned int irq, cpumask_t dest);
49 };
-----
```

第 41 行：typename 用于存放 PIC (Programmable Interrupt Controller, 可编程中断控制器) 的名称 (PIC 将在后面详细介绍)。

第 42~48 行：这些字段用于存放一些指向 PIC 特有的编程函数的指针。

现在，来看看 PPC 的中断控制器吧。首先看看 PowerMac 的 PIC：

```
-----
arch/ppc/platforms/pmac_pic.c
170 struct hw_interrupt_type pmac_pic = {
```

```

171  " PMAC-PIC ",
172  NULL,
173  NULL,
174  pmac_unmask_irq,
175  pmac_mask_irq,
176  pmac_mask_and_ack_irq,
177  pmac_end_irq,
178  NULL
179  };

```

可以看到，这个 PIC 的名字是 PMAC-PIC，它定义了 6 个函数中的 4 个。其中，`pmac_unmask_irq` 和 `pmac_mask_irq` 函数分别用于启用和禁用 IRQ 线，函数 `pmac_mask_and_ack_irq` 用于确认 IRQ 已收到，函数 `pmac_end_irq` 则负责中断处理程序执行完后的清理工作。

```

-----
arch/i386/kernel/i8259.c
59  static struct hw_interrupt_type i8259A_irq_type = {
60  "XT-PIC",
61  startup_8259A_irq,
62  shutdown_8259A_irq,
63  enable_8259A_irq,
64  disable_8259A_irq,
65  mask_and_ack_8259A,
66  end_8259A_irq,
67  NULL
68  };

```

x86 的 8259 PIC 被称为 XT-PIC，它定义了最前面的 5 个函数。前两个函数 `startup_8259A_irq` 和 `shutdown_8259A_irq`，分别用于启动和关闭实际的 IRQ 线。

3. 中断示例：系统定时器

系统定时器是操作系统的脉搏。系统启动时，系统定时器及其中断在系统初始化期间被初始化。此时对中断进行初始化所使用的接口与运行时中断被注册所使用的接口不同。接下来分析这个示例时将指出这两者的差异。

因为有更复杂的芯片生产出来，内核设计者对于系统定时器的来源多了几种选择。对 x86 体系结构来说，最常见的定时器实现是 PIT (Programmable Interval Timer, 可编程间隔定时器)，对 PowerPC 来说，最常见的定时器是衰减器。

x86 体系结构曾经用 Intel 8254 定时器实现 PIT。8254 用做 16 位的递减计数器——终端计数中断，即在寄存器中写入一个数值，8254 递减这个值，直到它变成 0。此时，它将激活 8259 中断控制器上的 IRQ 0 输中断，IRQ 0 在本节前面已经提到过。

在 PowerPC 体系结构中实现的系统定时器是衰减定时器，它是一个 32 位的递减计数器，且与 CPU 相同的频率运行。和 8259 类似，它将在达到其终止计数时激活一个中断。和 Intel 体系结构不同的是，衰减器被嵌入处理器中。

每当系统定时器递减，并激活一个中断，就称为一个节拍 (tick)。这个节拍的速率或频率由 `HZ` 变量设置。

HZ

HZ 是 Hertz 缩写 (Hz) 的变种, 因 Heinrich Hertz (1857—1894) 而命名。作为无线电波的创始人之一, 赫兹通过 Y 线环感应一个电火花证明了麦克斯韦关于电磁学的理论。然后, 马可尼在这些实验的基础上创立了现代无线电。为了向赫兹和他的成果表示敬意, 频率的基本单位就以他的名字来命名; 每秒 1 个周期等于 1 赫兹。

HZ 是在 `include/asm-xxx/param.h` 中定义的。让我们来看看这些值在 x86 和 PPC 中分别是多少。

```
-----
include/asm-i386/param.h
005 #ifdef __KERNEL__
006 #define HZ          1000      /* 内核内部定时器频率 */
kernel timer frequenay */
-----
include/asm-ppc/param.h
008 #ifdef __KERNEL__
009 #define HZ          100        /* 内核内部定时器频率 */
kernel timer frequenay */
-----
```

在大多数体系结构中, HZ 的值通常是 100, 但是因为机器的运行速率变得越来越快, 节拍率以某种模式增大了。本书中给出的两种主要体系结构的默认节拍速率都是 1 000。1 个节拍的周期是 $1/\text{HZ}$ 。这样, 周期 (或中断的间隔时间) 就是 1 毫秒。当 HZ 的值增大时, 将会在给定的时间里获得更多中断。尽管从计时功能来看这是更优的解决方案, 但是要注意重要的一点: 更多的处理器时间花费在应答内核的系统时钟中断上了。极端地说, 这可能减慢系统对用户态程序的响应。由于需要对所有的中断进行处理, 因此, 在两者之间找到恰当的平衡点是至关重要的。

现在开始分析系统定时器及其相关中断的初始化代码。系统定时器的处理程序是在内核初始化临近结束的时候才安装的。首先来看看代码段中的 `start_kernel()`, 这是系统引导时执行的主要初始化函数, 它首先调用 `trap_init()`, 接着调用 `init_IRQ()`, 最后再调用 `time_init()`:

```
-----
init/main.c
386 asmlinkage void __init start_kernel(void)
387 {
...
413 trap_init();
...
415 init_IRQ();
...
419 time_init();
...
}
```

第 413 行: 对于运行在保护模式下的 x86 体系结构, 宏 `trap_init()` 在中断描述符表 (IDT) 中初始化异常的入口。IDT 是在内存中设立的一张表。它的地址是在处理器的 IDTR 寄存器中设

置的。它的每个元素都是三种门当中的某一种。门是 x86 保护模式下的地址，由选择子、偏移量和特权级组成，其用途是转移程序的控制权。IDT 中有三种门，即系统门、中断门和陷阱门；所谓系统门就是把控制权转移给另一个进程；所谓中断门，就是在中断禁用的情况下，把控制权传递给中断处理程序；所谓陷阱门，就是在中断不改变的情况下，把控制权传递给中断处理程序。

PPC 被设计成跳转到一个特定地址这个地址依赖于异常。函数 `trap_init()` 对 PPC 来说是一个空操作。稍后继续分析系统定时器的代码时，将把 PPC 的中断描述符表和下面将要初始化的 x86 中断描述符表进行对比。

```
-----
arch/i386/kernel/traps.c
900 void __init trap_init(void)
901 {
902     #ifdef CONFIG_EISA
903         if (isa_readl(0x0FFFD9) == 'E' + ('I' << 8) + ('S' << 16) + ('A' << 24)) {
904             EISA_bus = 1;
905         }
906     #endif
907
908     #ifdef CONFIG_X86_LOCAL_APIC
909         init_apic_mappings();
910     #endif
911
912     set_trap_gate(0, &divide_error);
913     set_intr_gate(1, &debug);
914     set_intr_gate(2, &nmi);
915     set_system_gate(3, &int3); /* int3-5 can be called from all */
916     set_system_gate(4, &overflow);
917     set_system_gate(5, &bounds);
918     set_trap_gate(6, &invalid_op);
919     set_trap_gate(7, &device_not_available);
920     set_task_gate(8, GDT_ENTRY_DOUBLEFAULT_TSS);
921     set_trap_gate(9, &coprocessor_segment_overrun);
922     set_trap_gate(10, &invalid_TSS);
923     set_trap_gate(11, &segment_not_present);
924     set_trap_gate(12, &stack_segment);
925     set_trap_gate(13, &general_protection);
926     set_intr_gate(14, &page_fault);
927     set_trap_gate(15, &spurious_interrupt_bug);
928     set_trap_gate(16, &coprocessor_error);
929     set_trap_gate(17, &alignment_check);
930     #ifdef CONFIG_X86_MCE
931         set_trap_gate(18, &machine_check);
932     #endif
933     set_trap_gate(19, &simd_coprocessor_error);
934
935     set_system_gate(SYSCALL_VECTOR, &system_call);
936
937     /*
938     * default LDT is a single-entry callgate to lcall7 for iBCS
939     * and a callgate to lcall27 for Solaris/x86 binaries
940     */
941     set_call_gate(&default_ldt[0], lcall7);
942     set_call_gate(&default_ldt[4], lcall27);
943
```

```

944  /*
945  * Should be a barrier for any external CPU state.
946  */
947  cpu_init();
948
949  trap_init_hook();
950  }

```

第 902 行：查找 EISA 签名。isa_readl() 是一个辅助程序，它通过用 ioremap() 映射 I/O 来读取 EISA 总线。

第 908~910 行：如果存在一个高级可编程中断控制器 (APIC) 的话，就把它的地址添加到系统固定的地址映射表。对于“特殊的”系统地址辅助例程 set_fixmap_nocache()，请参阅文件 include/asm-i386/fixmap.h。init_apic_mappings() 使用这个例程来设置 APIC 的物理地址。

第 912~935 行：用陷阱门、系统门和中断门来初始化 IDT。

第 941~942 行：这些特殊的段间调用门来支持 Intel 的二进制兼容标准 (Intel Binary Compatibility Standard)，这个标准允许在 Linux 上运行其他的 UNIX 二进制程序。

第 947 行：初始化当前正在执行的 CPU 的中断描述符表和寄存器。

第 949 行：用于初始化系统特定的硬件，例如不同种类的 APIC。这对于大多数 x86 平台而言是一个空操作。

第 415 行^①：调用 init_IRQ() 来初始化硬件中断控制器。x86 和 PPC 体系结构都拥有多个设备的实现，对于 x86 体系结构，我们分析 i8259 设备。对于 PPC，分析和 Power Mac 相关的代码。

PPC 中 init_IRQ() 是在 arch/ppc/kernel/irq.c 中实现的。该函数根据特殊的硬件配置调用对应的某个例程来初始化 PIC。对于 Power Mac 配置而言，调用 arch/ppc/platforms/pmac_pic.c 中的函数 pmac_pic_init() 来初始化 G3、G4 和 G5 的 I/O 控制器。这是一个硬件特有的例程，它将设法识别 I/O 控制器的类型并对其进行适当的配置。在本例中，PIC 是 I/O 中断控制器设备的一部分，其中断初始化过程与 x86 类似，微小的区别在于，系统定时器不是在 PPC 的 init_IRQ() 函数而是在 time_init() 函数中启动的，本节稍后将介绍 time_init() 函数。

x86 体系结构对于 PIC 的选择比较少。如前所述，早期的系统使用级联的 8259，后来则使用 IOAPIC 体系结构。下面这段代码分析带有模拟 8259 中断控制器的 APIC：

```

-----
arch/i386/kernel/i8259.c
342 void __init init_ISA_irqs (void)
343 {
344     int i;
345     #ifdef CONFIG_X86_LOCAL_APIC
346     init_bsp_APIC();
347     #endif
348     init_8259A(0);
349     ...
351     for (i = 0; i < NR_IRQS; i++) {
352         irq_desc[i].status = IRQ_DISABLED;

```

① 原文中没有给出这一行代码，在内核源码中查找后，发现是位于 /init/main.c 中的 start_kernel() 调用了这个函数。